

在 TPU 使用 Keras 和新式 convnets

來源：<https://codelabs.developers.google.com/codelabs/keras-flowers-tpu?hl=zh-tw>

步驟數：13

在本研究室中，您將瞭解如何從頭開始建構、訓練及調整捲積類神經網路。現在只要使用 TPU 的強大功能，就能在幾分鐘內完成這項作業。您也將探索多種做法，包括非常簡單的遷移學習，以及 Squeezenet 等現代卷積架構。本研究室包含有關類神經網路的必要理論解釋，也是開發人員學習深度學習的好起點。本研究室使用 Tensorflow 2。

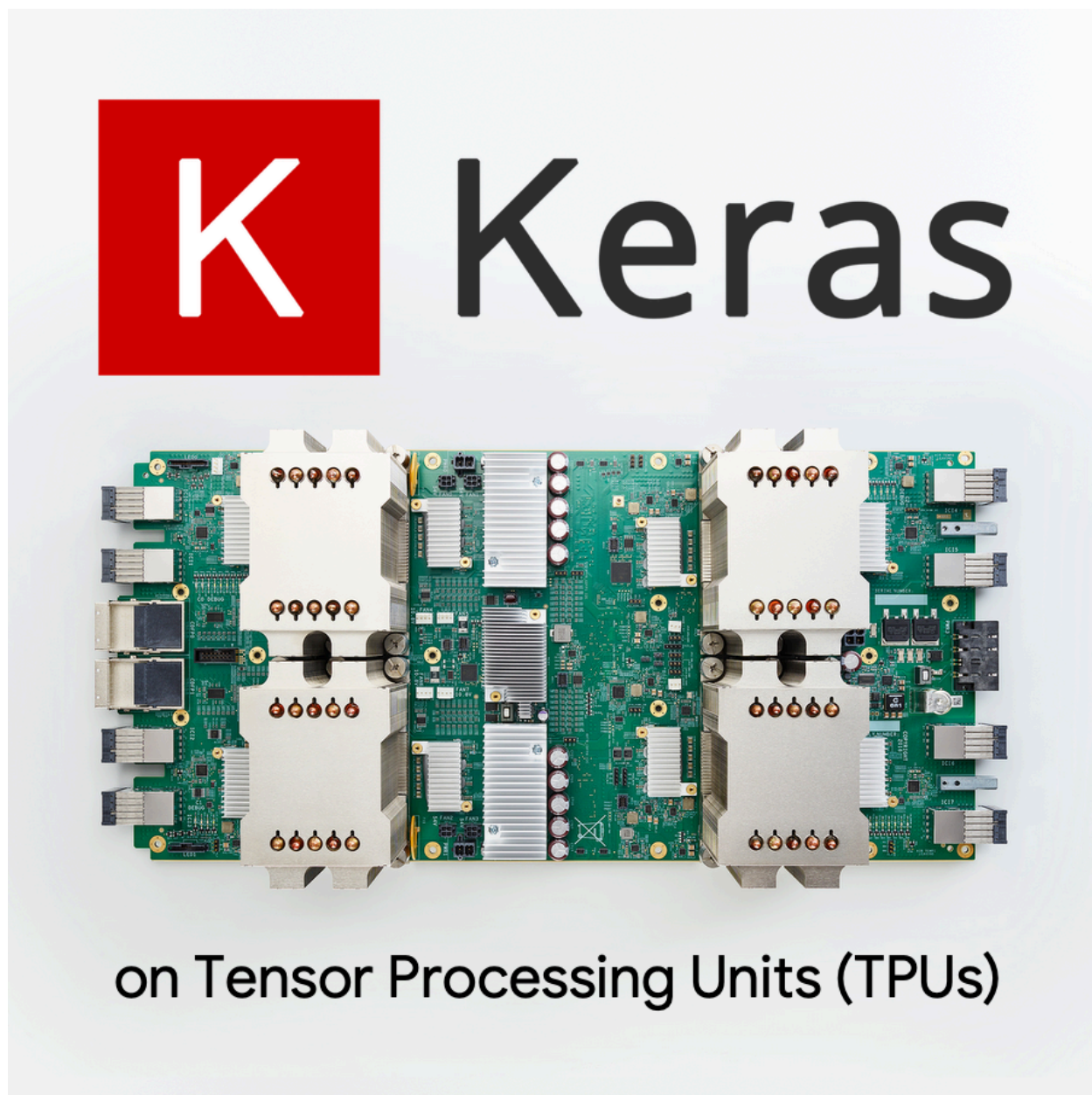
目錄

1. 總覽
2. Google Colaboratory 快速入門
3. [INFO] 什麼是 Tensor Processing Unit (TPU) ?
4. 正在載入資料
5. 快速載入資料
6. [INFO] Neural network classifier 101
7. 遷移學習
8. [INFO] 卷積類神經網路
9. 您的自訂轉換網路
10. [INFO] 現代卷積架構
11. Squeezenet
12. 微調 Xception
13. 恭喜 !

1. 總覽

在本實驗室中，您將瞭解如何使用 Keras 和 TensorFlow 2，從頭開始建構、訓練及調整自己的卷積類神經網路。現在，只要運用 TPU 的強大效能，幾分鐘就能完成這項作業。您也會瞭解多種方法，從非常簡單的遷移學習到 Squeezenet 等現代卷積架構。這項實驗室包含類神經網路的理論說明，是開發人員學習深度學習的良好起點。

閱讀深度學習論文可能很困難，也容易感到困惑。現在就來實際瞭解現代卷積類神經網路架構。



課程內容

- 使用 Keras 和 Tensor Processing Unit (TPU) 更快速地建構自訂模型。
- 使用 `tf.data.Dataset` API 和 TFRecord 格式，有效率地載入訓練資料。
- 如要作弊 🐱，請使用遷移學習，而不是自行建構模型。
- 使用 Keras 序列和函式模型樣式。
- 使用 softmax 層和交叉熵損失，建構自己的 Keras 分類器。
- 使用合適的捲積層微調模型。

- 探索模組、全域平均集區化等現代 convnet 架構概念。
- 使用 Squeezenet 架構建構簡單的現代 convnet。

意見回饋

如果您發現這個程式碼研究室有任何錯誤，請告訴我們。你可以透過 GitHub 問題 [[意見回饋連結](#)] 提供意見。

步驟 2 · 約 4 分鐘

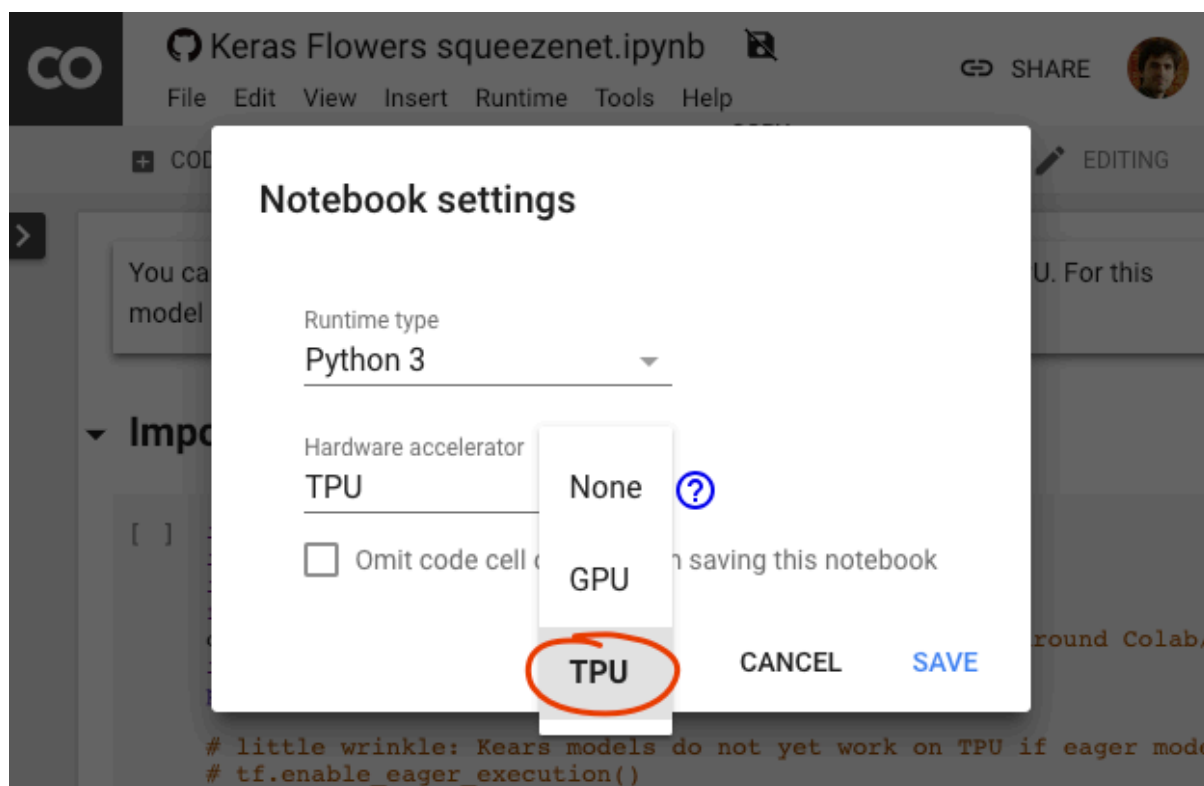
2. Google Colaboratory 快速入門

本實驗室使用 Google Colaboratory，您無須進行任何設定。您可以在 Chromebook 上執行這項工具。請開啟下方檔案並執行儲存格，熟悉 Colab 筆記本。



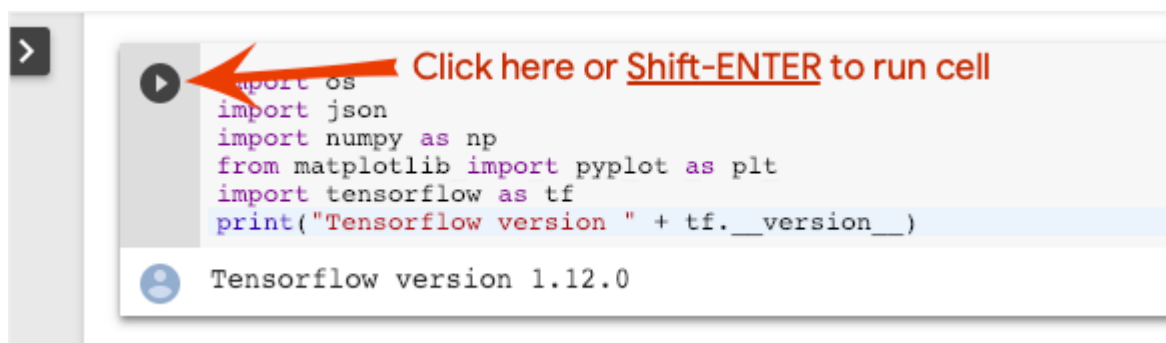
Welcome to Colab.ipynb

選取 TPU 後端



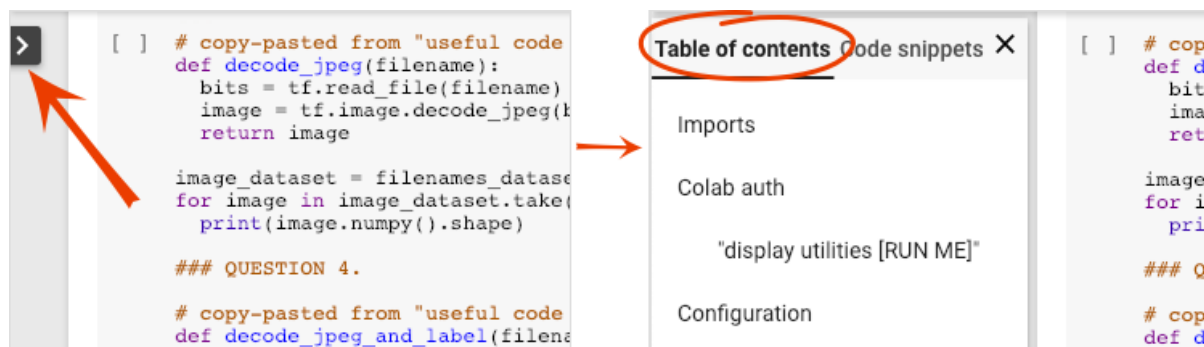
在 Colab 選單中，依序選取「執行階段」>「變更執行階段類型」，然後選取「TPU」。在本程式碼研究室中，您將使用強大的 TPU (Tensor Processing Unit)，支援硬體加速訓練。首次執行時，系統會自動連線至執行階段，您也可以使用右上角的「連線」按鈕。

筆記本執行



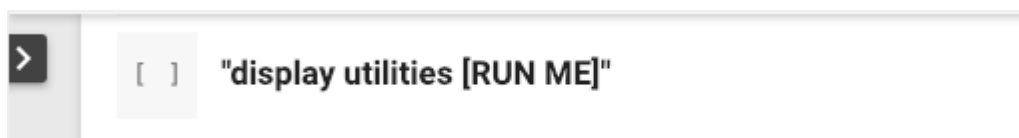
按一下儲存格並使用 Shift-ENTER，即可一次執行一個儲存格。您也可以依序選取「執行階段」>「全部執行」，執行整個筆記本。

Table of contents



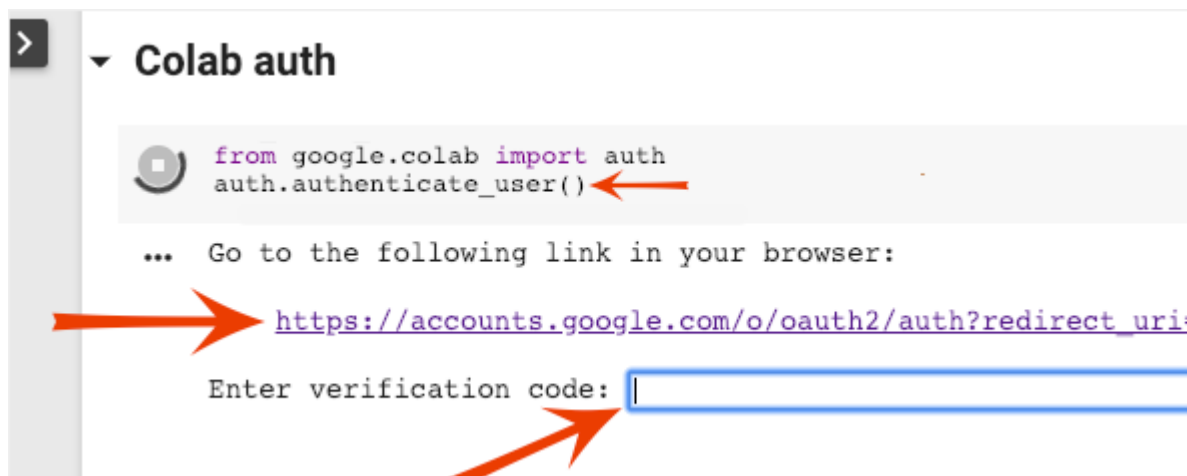
所有筆記本都有目錄。你可以使用左側的黑色箭頭開啟這個選單。

隱藏的儲存格



部分儲存格只會顯示標題，這是 Colab 專屬的筆記本功能。您可以按兩下查看其中的程式碼，但通常不會很有趣。通常是支援或視覺化函式。您仍須執行這些儲存格，才能定義其中的函式。

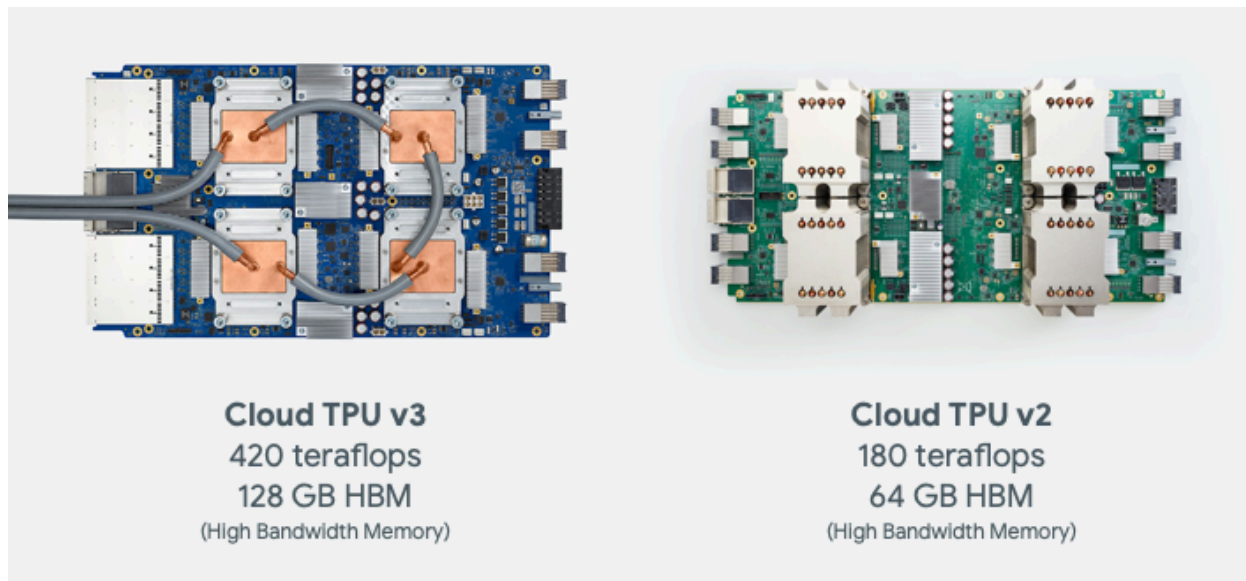
驗證



只要使用已授權的帳戶進行驗證，Colab 就能存取您的私人 Google Cloud Storage bucket。上述程式碼片段會觸發驗證程序。

3. [INFO] 什麼是 Tensor Processing Unit (TPU) ?

簡介



TPU 是專門用於深度學習任務的硬體加速器。在本程式碼研究室中，您將瞭解如何搭配 Keras 和 TensorFlow 2 使用這些工具。Cloud TPU 的基本設定為 8 個核心，此外還有多種大型設定，稱為「TPU Pod」，最多可達 2048 個核心。增加訓練批量，藉此加速訓練。

在 Keras 中於 TPU 上訓練模型的程式碼 (如果沒有 TPU，則改用 GPU 或 CPU)：

```
try: # detect TPUs
    tpu = tf.distribute.cluster_resolver.TPUClusterResolver.connect()
    strategy = tf.distribute.TPUStrategy(tpu)
except ValueError: # detect GPUs
    strategy = tf.distribute.MirroredStrategy() # for CPU/GPU or multi-GPU machines

# use TPUStrategy scope to define model
with strategy.scope():
    model = tf.keras.Sequential( ... )
    model.compile( ... )

# train model normally on a tf.data.Dataset
model.fit(training_dataset, epochs=EPOCHS, steps_per_epoch=...)
```

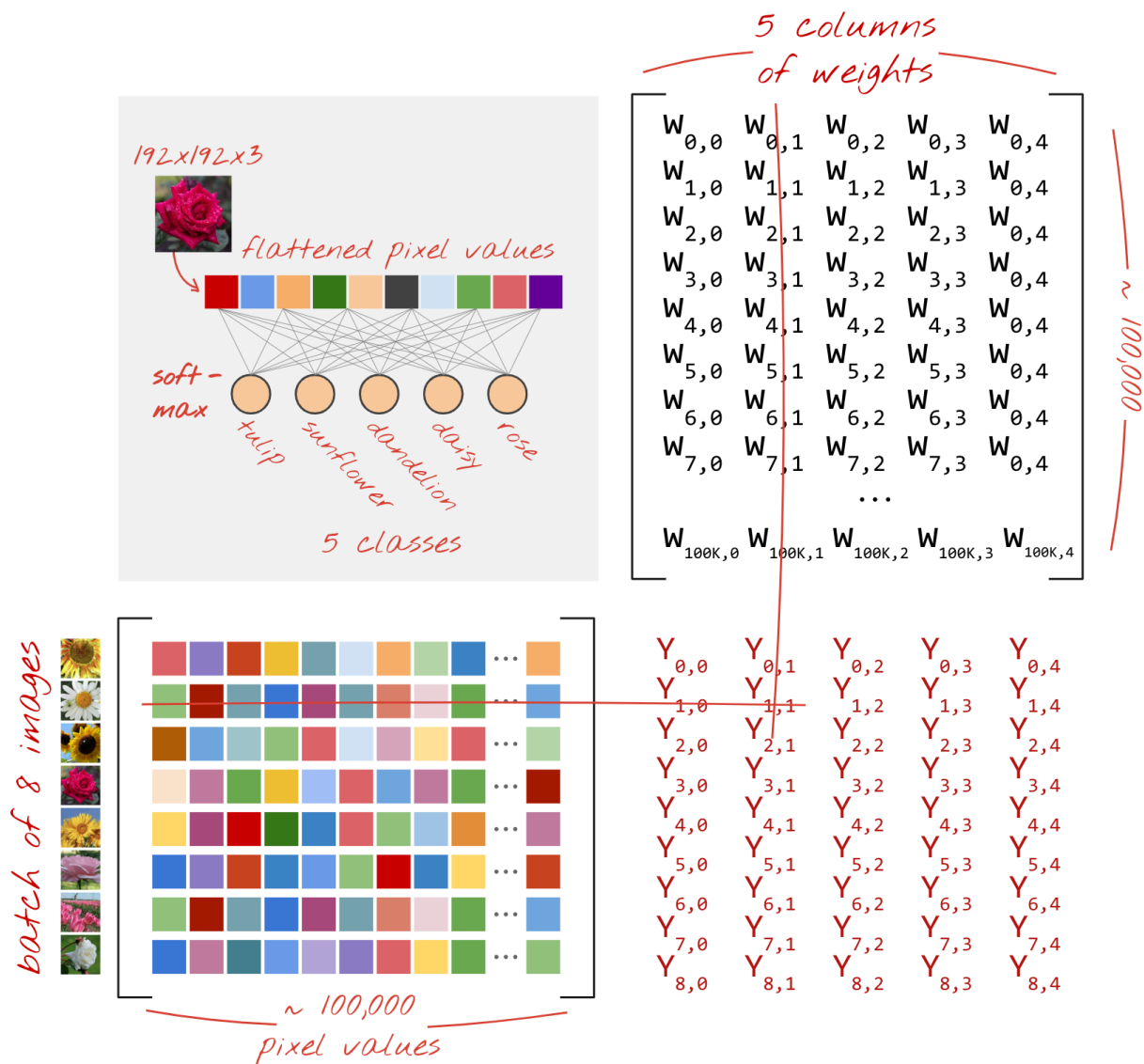
我們今天將使用 TPU，以互動式速度 (每次訓練執行幾分鐘) 建構及最佳化花朵分類器。

If this is old news to you **SKIP**  to the next exercise otherwise **READ ON**



為何選擇 TPU ?

現代 GPU 是以可程式化的「核心」為基礎架構，這種架構非常靈活，可處理各種工作，例如 3D 算繪、深度學習、物理模擬等。另一方面，TPU 會將傳統向量處理器與專用矩陣乘法單元配對，並擅長處理大型矩陣乘法運算占主導地位的任何工作，例如類神經網路。

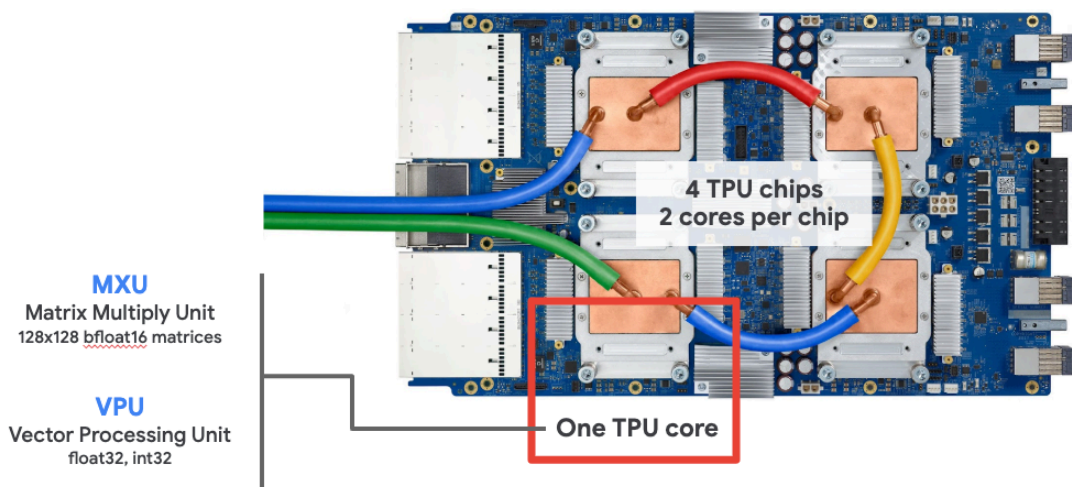


插圖：密集類神經網路層，以矩陣乘法表示，一次透過類神經網路處理一批八張圖片。請執行一行 x 欄的乘法，確認這確實是圖像所有像素值的加權總和。雖然有點複雜，但卷積層也可以表示為矩陣乘法 (請參閱第 1 節的說明)。

硬體

MXU 和 VPU

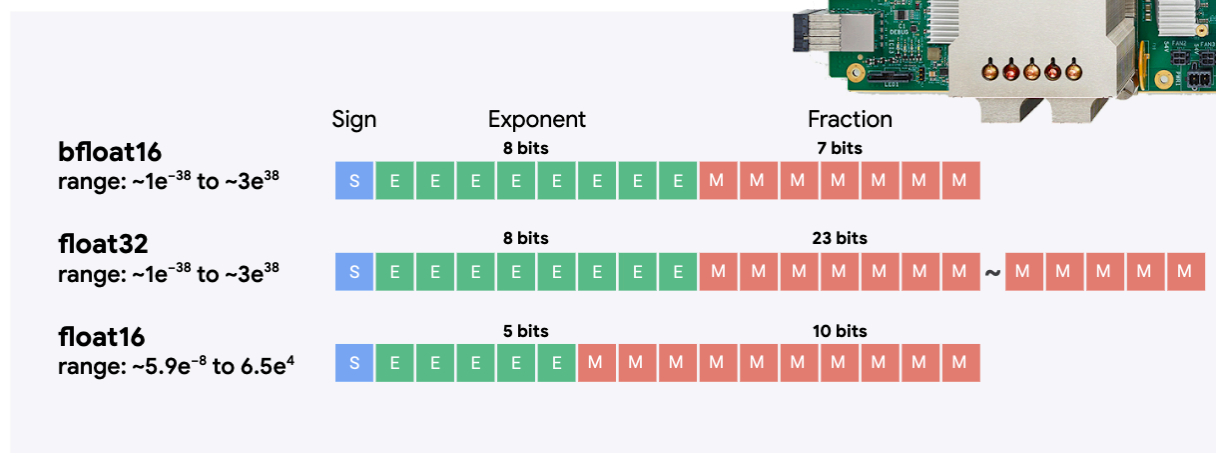
TPU v2 核心由矩陣乘法單元 (MXU) 和向量處理單元 (VPU) 組成，前者用於執行矩陣乘法，後者則用於執行所有其他工作，例如活化、softmax 等。VPU 會處理 float32 和 int32 運算。另一方面，MXU 則是以混合精度 16-32 位元浮點格式運作。



混合精確度浮點和 bfloat16

MXU 會使用 bfloat16 輸入和 float32 輸出計算矩陣乘法。中繼累計作業會以 float32 精確度執行。

bfloat16



類神經網路訓練通常能抵禦因浮點數精確度降低而產生的雜訊。在某些情況下，雜訊甚至有助於最佳化工具收斂。傳統上，16 位元浮點數精度用於加速運算，但 float16 和 float32 格式的範圍差異很大。將精確度從 float32 降為 float16 通常會導致溢位和下溢。雖然有解決方案，但通常需要額外作業才能讓 float16 正常運作。

因此，Google 在 TPU 中導入了 bfloat16 格式。bfloat16 是經過截斷的 float32，與 float32 的指數位元和範圍完全相同。此外，TPU 會以混合精度計算矩陣乘法，並使用 bfloat16 輸入，但輸出為 float32，因此通常不需要變更程式碼，就能享有精度降低帶來的效能提升。

在 TPU 上，預設會使用 bfloat16/float32 混合精確度。您不必變更 Tensorflow 程式碼，即可啟用這項功能。您仍可在整個程式碼中使用 float32。執行矩陣乘法時，TPU 會自動將輸入內容截斷為 bfloat16。產生的矩陣為 float32。

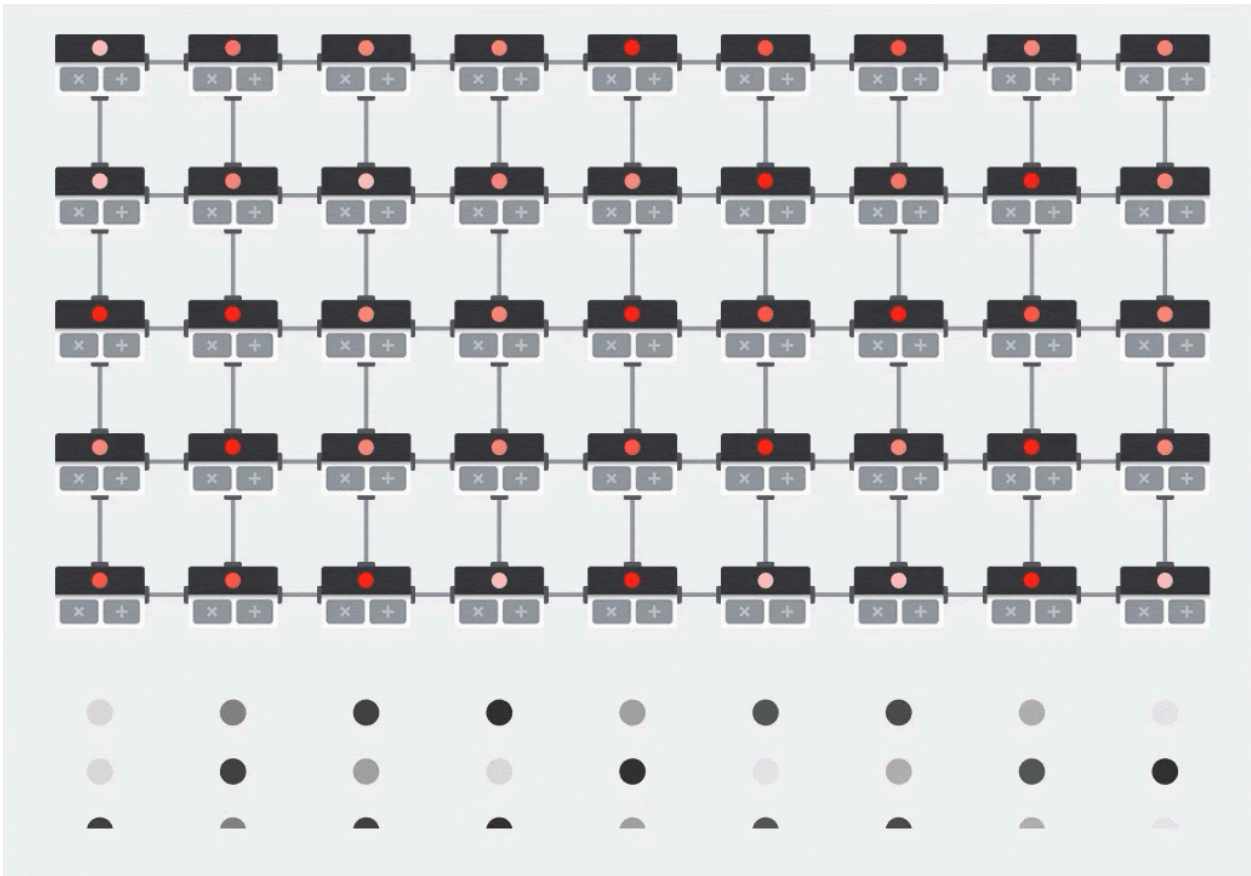
Systolic array

MXU 會使用所謂的「脈動陣列」架構，在硬體中實作矩陣乘法，讓資料元素流經硬體運算單元陣列。(在醫學上，「收縮」是指心臟收縮和血流，這裡則是指資料流動。)

矩陣乘法的基本元素是兩個矩陣中，一個矩陣的列與另一個矩陣的欄之間的點積 (請參閱本節頂端的插圖)。以矩陣乘法 $Y=X*W$ 來說，結果的一個元素會是：

$$Y[2,0] = X[2,0]*W[0,0] + X[2,1]*W[1,0] + X[2,2]*W[2,0] + \dots + X[2,n]*W[n,0]$$

在 GPU 上，您可以將這個點積程式設計到 GPU「核心」，然後在盡可能多的「核心」上平行執行，嘗試一次計算結果矩陣的每個值。如果產生的矩陣為 128x128 大小，則需要 128x128=16K 個「核心」，這通常是不可能的。最大的 GPU 約有 4000 個核心。另一方面，TPU 會為 MXU 中的運算單元使用最少的硬體：只有 `bfloat16 x bfloat16 => float32` 乘法累加器，沒有其他元件。這些單元非常小，TPU 可以在 128x128 MXU 中實作 16K 個單元，並一次處理這個矩陣乘法。

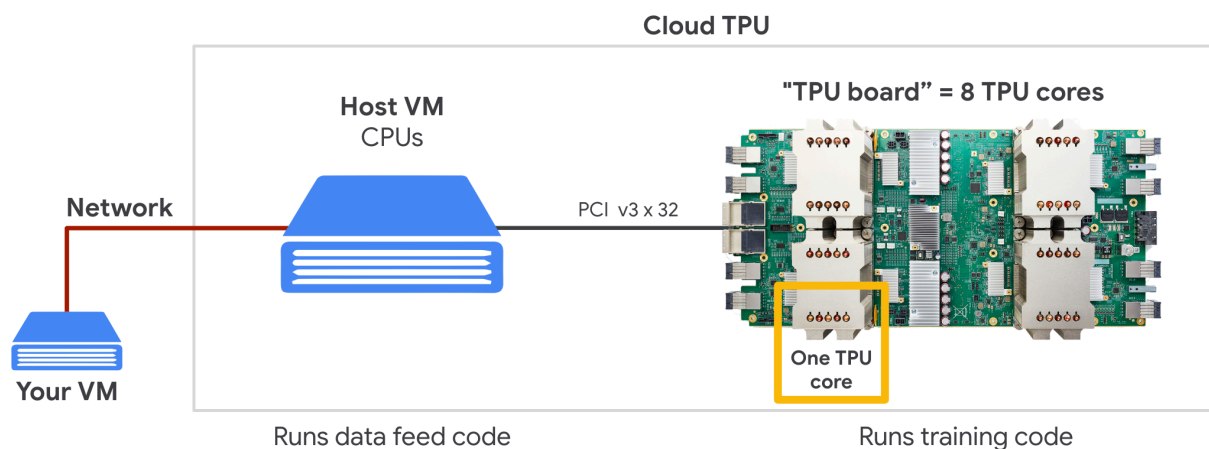


插圖：MXU 脈動陣列。運算元素是乘法累加器。一個矩陣的值會載入陣列 (紅點)。其他矩陣的值會流經陣列 (灰色點)。垂直線會將值向上傳播。水平線會傳播部分總和。使用者可自行驗證，當資料流經陣列時，右側會輸出矩陣乘法運算的結果。

此外，在 MXU 中計算點積時，中間總和只會在相鄰的運算單元之間流動。這類值不需要儲存及從記憶體或暫存器檔案中擷取。因此，在計算矩陣乘法時，TPU 脈動陣列架構在密度和電力方面具有顯著優勢，且速度也比 GPU 快上不少。

Cloud TPU

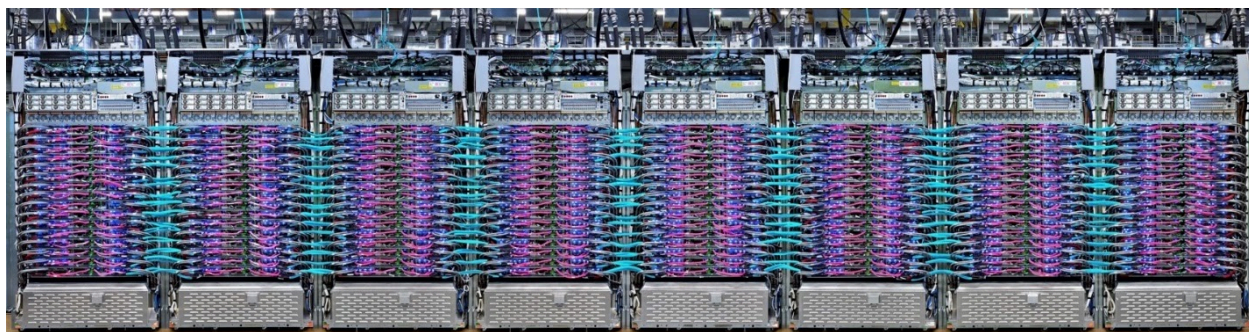
在 Google Cloud Platform 上要求「[Cloud TPU v2](#)」時，您會取得具有 PCI 連接 TPU 板的虛擬機器 (VM)。TPU 板有四個雙核心 TPU 晶片。每個 TPU 核心都具備 VPU (向量處理單元) 和 128x128 MXU (矩陣乘法單元)。這個「Cloud TPU」通常會透過網路連線至要求該 TPU 的 VM。因此，完整圖片如下所示：



插圖：您的 VM，以及透過網路連結的「Cloud TPU」加速器。「Cloud TPU」本身是由 VM 和 PCI 連接的 TPU 板組成，板上則有四個雙核心 TPU 晶片。

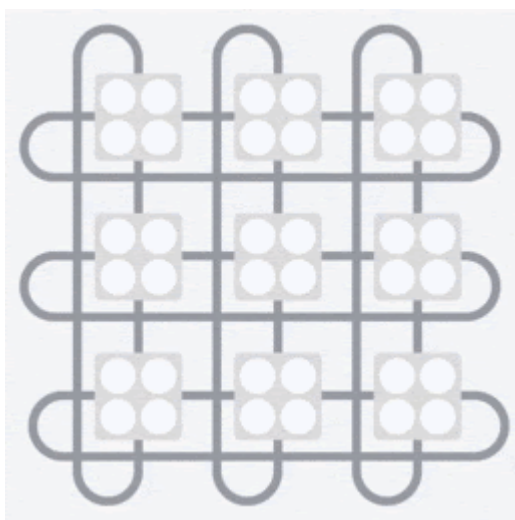
TPU Pod

在 Google 的資料中心，TPU 會連線至高效能運算 (HPC) 互連，因此可視為一個非常大的加速器。Google 將這些節點稱為 Pod，最多可包含 512 個 TPU v2 核心或 2048 個 TPU v3 核心。



插圖：TPU v3 Pod。透過 HPC 互連網路連線的 TPU 板和機架。

在訓練期間，系統會使用 all-reduce 演算法在 TPU 核心之間交換梯度 ([這裡有 all-reduce 的詳細說明](#))。訓練中的模型可透過訓練大量批次，充分運用硬體資源。



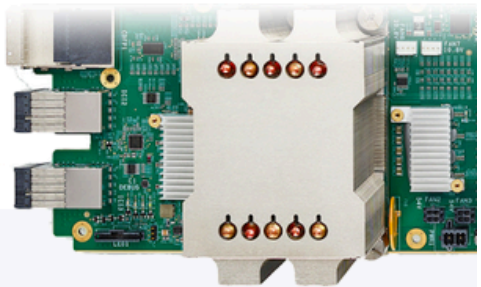
插圖：在 Google TPU 的 2D 環面網格 HPC 網路中，使用 all-reduce 演算法訓練期間的梯度同步。

軟體

大型批量訓練

TPU 的理想批量大小為每個 TPU 核心 128 個資料項目，但每個 TPU 核心 8 個資料項目時，硬體就能展現良好的使用率。請注意，一個 Cloud TPU 有 8 個核心。

在本程式碼研究室中，我們將使用 Keras API。在 Keras 中，您指定的批次是整個 TPU 的全域批量大小。系統會自動將批次分割為 8 個，並在 TPU 的 8 個核心上執行。



TPU batch size
ideal 128 per TPU core
interesting from 8 per core

DONE!
scales from
8 to 2048 cores

如需其他效能提示，請參閱 [TPU 效能指南](#)。如果批次大小非常大，部分模型可能需要特別注意，詳情請參閱 [LARSOptimizer](#)。

幕後花絮：XLA

Tensorflow 程式會定義運算圖形。TPU 不會直接執行 Python 程式碼，而是執行 TensorFlow 程式定義的運算圖。在幕後，名為 XLA (加速線性代數編譯器) 的編譯器會將運算節點的 Tensorflow 圖形轉換為 TPU 機器碼。這個編譯器也會對程式碼和記憶體配置執行許多進階最佳化作業。當工作傳送至 TPU 時，系統會自動進行編譯。您不必在建構鏈中明確加入 XLA。



TensorFlow graph

```
def model():  
    y = tf.add(x, 1)  
    tf.reduce_sum(y)
```

XLA representation

```
broadcast.4.133 = f32[100,25]{1,0} broadcast  
transpose.4.129 = f32[100,32]{1,0} transpos  
dot.4.130 = f32[100,25]{1,0} dot(transpose.  
cross-replica-sum.4.131 = f32[100,25]{1,0}  
multiply.4.134 = f32[100,25]{1,0} multiply(  
subtract.4.135 = f32[100,25]{1,0} subtract(  
reshape.4.136 = f32[2500]{0} reshape(subtra
```

TPU machine code

```
89 50 4E 47 0D 0A 1A 0A 00 00 00 00 49 48 44 52 00 00  
03 C0 00 00 02 1C 08 02 00 00 00 B6 3F 50 2C 00 00 80  
00 49 44 41 54 78 DA EC 9D 67 58 1C 57 96 F7 E7 D9 F7  
C3 EE B3 FB 65 BF EC F3 EC 6C 98 D9 19 D9 33 4E B3 63  
8F 67 76 3C 1E DB 33 63 7B 6C 2B 21 14 10 12 28 E7 2C  
A1 2C 2B 23 10 20 24 91 33 88 20 40 12 39 67 10 49 E4  
8C 40 84 6E 68 72 EE 6E A0 C9 B2 F4 FE AB 8F 54 6E 01  
42 28 80 00 9D E3 A3 F2 A5 EA D6 AD EA AA 1B 7E F7 D6  
B9 E7 FE A4 FA C2 2F 59 59 59 59 59 59 59 59 59 59
```

TF to XLA bridge

XLA compiler

圖解：如要在 TPU 上執行，Tensorflow 程式定義的運算圖會先轉換為 XLA (加速線性代數編譯器) 表示法，然後由 XLA 編譯為 TPU 機器碼。

在 Keras 中使用 TPU

Tensorflow 2.1 以上版本支援透過 Keras API 使用 TPU。Keras 支援 TPU 和 TPU Pod。以下範例適用於 TPU、GPU 和 CPU：

```
try: # detect TPUs
    tpu = tf.distribute.cluster_resolver.TPUClusterResolver.connect()
    strategy = tf.distribute.TPUStrategy(tpu)
except ValueError: # detect GPUs
    strategy = tf.distribute.MirroredStrategy() # for CPU/GPU or multi-GPU machines

# use TPUStrategy scope to define model
with strategy.scope():
    model = tf.keras.Sequential( ... )
    model.compile( ... )

# train model normally on a tf.data.Dataset
model.fit(training_dataset, epochs=EPOCHS, steps_per_epoch=...)
```

在這個程式碼片段中：

- `TPUClusterResolver().connect()` 會在網路上尋找 TPU。在大多數 Google Cloud 系統 (AI Platform 工作、Colaboratory、Kubeflow、透過「ctpu up」公用程式建立的深度學習 VM) 上，這項工具都能在沒有參數的情況下運作。這些系統會透過 `TPU_NAME` 環境變數得知 TPU 所在位置。如要手動建立 TPU，請在您使用的 VM 上設定 `TPU_NAME` 環境變數，或使用明確的參數呼叫 `TPUClusterResolver`：`TPUClusterResolver(tp_uname, zone, project)`
- `TPUStrategy` 是實作分配和「全縮減」梯度同步演算法的部分。
- 這項策略是透過範圍套用。模型必須在策略範圍() 內定義。
- `tpu_model.fit` 函式預期會收到 `tf.data.Dataset` 物件，做為 TPU 訓練的輸入內容。

常見的 TPU 移植作業

- 雖然在 TensorFlow 模型中載入資料的方法有很多種，但使用 TPU 時，必須使用 `tf.data.Dataset` API。
- TPU 的速度非常快，因此在 TPU 上執行時，擷取資料通常會成為瓶頸。您可以使用 [TPU 效能指南](#) 中的工具，偵測資料瓶頸和其他效能提示。
- 系統會將 `int8` 或 `int16` 數字視為 `int32`。TPU 沒有以少於 32 位元運作的整數硬體。
- 系統不支援部分 TensorFlow 作業。[請參閱這份清單](#)。好消息是，這項限制只適用於訓練程式碼，也就是透過模型的前向和後向傳遞。您仍可在資料輸入管道中使用所有 TensorFlow 作業，因為這些作業會在 CPU 上執行。
- TPU 不支援 `tf.py_func`。

步驟 4 · 約 20 分鐘

4. 正在載入資料





799 Tulips
699 Sunflowers
641 Roses
633 Daisies
898 Dandelions

3670 flowers

我們將使用花卉圖片資料集。目標是學會將這些花朵分類為 5 種花卉類型。資料載入作業是使用 `tf.data.Dataset` API 執行。首先，請先瞭解 API。

實作實驗室

請開啟下列筆記本、執行儲存格 (Shift-ENTER)，並按照標示「WORK REQUIRED」的指示操作。



[Fun with tf.data.Dataset \(playground\).ipynb](#)

其他資訊

關於「flowers」資料集

資料集分為 5 個資料夾。每個資料夾都包含一種花朵。資料夾名稱分別為向日葵、雛菊、蒲公英、鬱金香和玫瑰。資料託管在 Google Cloud Storage 的公開 bucket 中。摘錄：

```
gs://flowers-public/sunflowers/5139971615_434ff8ed8b_n.jpg
gs://flowers-public/daisy/8094774544_35465c1c64.jpg
gs://flowers-public/sunflowers/9309473873_9d62b9082e.jpg
gs://flowers-public/dandelion/19551343954_83bb52f310_m.jpg
gs://flowers-public/dandelion/14199664556_188b37e51e.jpg
gs://flowers-public/tulips/4290566894_c7f061583d_m.jpg
gs://flowers-public/roses/3065719996_c16ecd5551.jpg
gs://flowers-public/dandelion/8168031302_6e36f39d87.jpg
gs://flowers-public/sunflowers/9564240106_0577e919da_n.jpg
gs://flowers-public/daisy/14167543177_cd36b54ac6_n.jpg
```

為什麼要使用 `tf.data.Dataset` ？

Keras 和 TensorFlow 的所有訓練和評估函式都會接受資料集。在資料集中載入資料後，API 會提供所有適用於類神經網路訓練資料的常見功能：

```
dataset = ... # load something (see below)
dataset = dataset.shuffle(1000) # shuffle the dataset with a buffer of 1000
dataset = dataset.cache() # cache the dataset in RAM or on disk
dataset = dataset.repeat() # repeat the dataset indefinitely
dataset = dataset.batch(128) # batch data elements together in batches of 128
AUTOTUNE = tf.data.AUTOTUNE
dataset = dataset.prefetch(AUTOTUNE) # prefetch next batch(es) while training
```

如需成效提示和資料集最佳做法，請參閱[這篇文章](#)。請參閱[這份參考文件](#)。

`tf.data.Dataset` 基礎知識

資料通常會以多個檔案的形式提供，這裡是指圖片。您可以呼叫下列項目，建立檔案名稱資料集：

```
filenames_dataset = tf.data.Dataset.list_files('gs://flowers-public/**/*.jpg')
# The parameter is a "glob" pattern that supports the * and ? wildcards.
```

接著，您會將函式「對應」至每個檔案名稱，這通常會將檔案載入並解碼為記憶體中的實際資料：

```
def decode_jpeg(filename):
    bits = tf.io.read_file(filename)
    image = tf.io.decode_jpeg(bits)
    return image

image_dataset = filenames_dataset.map(decode_jpeg)
# this is now a dataset of decoded images (uint8 RGB format)
```

如要疊代資料集，請按照下列步驟操作：

```
for data in my_dataset:
    print(data)
```

元組資料集

在監督式學習中，訓練資料集通常由訓練資料和正確答案配對組成。如要允許這項操作，解碼函式可以傳回元組。接著，您會取得元組資料集，並在疊代時傳回元組。傳回的值是 Tensorflow 張量，可供模型使用。您可以對這些值呼叫 `.numpy()`，查看原始值：

```
def decode_jpeg_and_label(filename):
    bits = tf.read_file(filename)
    image = tf.io.decode_jpeg(bits)
    label = ... # extract flower name from folder name
    return image, label

image_dataset = filenames_dataset.map(decode_jpeg_and_label)
# this is now a dataset of (image, label) pairs

for image, label in dataset:
    print(image.numpy().shape, label.numpy())
```

結論：逐一載入圖片的速度很慢！

反覆處理這個資料集時，您會發現每秒可載入 1 到 2 張圖片。這太慢了！我們用於訓練的硬體加速器可維持這個速率的許多倍。請參閱下一節，瞭解如何達成這個目標。

解決方案

解決方案筆記本在此。如果遇到困難，可以參考這項工具。



[Fun with tf.data.Dataset \(solution\).ipynb](#)

涵蓋內容

- 🤔 `tf.data.Dataset.list_files`
- 🤔 `tf.data.Dataset.map`
- 🤔 元組資料集
- 😊 逐一查看資料集

請花點時間在腦中瀏覽這份檢查清單。

5. 快速載入資料

我們將在本實驗室中使用的 Tensor Processing Unit (TPU) 硬體加速器速度非常快。挑戰通常在於如何快速提供資料，讓這些加速器保持忙碌。Google Cloud Storage (GCS) 能夠維持非常高的處理量，但與所有雲端儲存系統一樣，啟動連線會產生一些網路來回傳輸費用。因此，將資料儲存為數千個個別檔案並非理想做法。我們將這些資料分批存入較少數量的檔案，並運用 `tf.data.Dataset` 的強大功能，平行讀取多個檔案。

朗讀

下列筆記本中的程式碼會載入圖片檔案、將圖片調整為相同大小，然後儲存在 16 個 TFRecord 檔案中。請快速瀏覽。由於本程式碼研究室的其餘部分會提供格式正確的 TFRecord 資料，因此不需要執行這項作業。



[Flower pictures to TFRecords.ipynb](#)

理想的資料版面配置，可達到最佳 GCS 處理量

經驗法則是將資料分割成數個 (數十到數百個) 較大的檔案 (數十到數百 MB)。如果檔案數量過多 (例如數千個)，存取每個檔案的時間可能會開始受到影響。如果檔案太少 (例如只有一或兩個)，就無法享有平行串流多個檔案的好處。

TFRecord 檔案格式

Tensorflow 偏好的資料儲存檔案格式是以 [protobuf](#) 為基礎的 TFRecord 格式。其他序列化格式也適用，但您可以編寫下列程式碼，直接從 TFRecord 檔案載入資料集：

```
filenames = tf.io.gfile.glob(FILENAME_PATTERN)
dataset = tf.data.TFRecordDataset(filenames)
dataset = dataset.map(...) # do the TFRecord decoding here - see below
```

為達到最佳效能，建議使用下列較複雜的程式碼，一次讀取多個 TFRecord 檔案。這段程式碼會平行讀取 N 個檔案，並忽略資料順序，以提高讀取速度。

```
AUTOTUNE = tf.data.AUTOTUNE
ignore_order = tf.data.Options()
ignore_order.experimental_deterministic = False

filenames = tf.io.gfile.glob(FILENAME_PATTERN)
dataset = tf.data.TFRecordDataset(filenames, num_parallel_reads=AUTOTUNE)
dataset = dataset.with_options(ignore_order)
dataset = dataset.map(...) # do the TFRecord decoding here - see below
```

TFRecord 一覽表

TFRecord 可儲存三種資料：**位元組字串** (位元組清單)、**64 位元整數**和 **32 位元浮點數**。這些值一律會儲存為清單，單一資料元素則會是大小為 1 的清單。您可以使用下列輔助函式，將資料儲存到 TFRecord。

寫入位元組字串

```
# warning, the input is a list of byte strings, which are themselves lists of bytes
def _bytestring_feature(list_of_bytestrings):
    return tf.train.Feature(bytes_list=tf.train.BytesList(value=list_of_bytestrings))
```

寫入整數

```
def _int_feature(list_of_ints): # int64
    return tf.train.Feature(int64_list=tf.train.Int64List(value=list_of_ints))
```

撰寫浮動文字

```
def _float_feature(list_of_floats): # float32
    return tf.train.Feature(float_list=tf.train.FloatList(value=list_of_floats))
```

編寫 TFRecord，使用上述輔助程式

```
# input data in my_img_bytes, my_class, my_height, my_width, my_floats
with tf.python_io.TFRecordWriter(filename) as out_file:
    feature = {
        "image": _bytestring_feature([my_img_bytes]), # one image in the list
        "class": _int_feature([my_class]),             # one class in the list
        "size": _int_feature([my_height, my_width]),   # fixed length (2) list of ints
        "float_data": _float_feature(my_floats)        # variable length list of floats
    }
    tf_record = tf.train.Example(features=tf.train.Features(feature=feature))
    out_file.write(tf_record.SerializeToString())
```

如要從 TFRecord 讀取資料，您必須先宣告所儲存記錄的版面配置。在宣告中，您可以將任何具名欄位當做固定長度清單或可變長度清單存取：

從 TFRecords 讀取資料

```
def read_tfrecord(data):
    features = {
        # tf.string = byte string (not text string)
        "image": tf.io.FixedLenFeature([], tf.string), # shape [] means scalar, here, a single
byte string
        "class": tf.io.FixedLenFeature([], tf.int64), # shape [] means scalar, i.e. a single
item
        "size": tf.io.FixedLenFeature([2], tf.int64), # two integers
        "float_data": tf.io.VarLenFeature(tf.float32) # a variable number of floats
    }

    # decode the TFRecord
    tf_record = tf.io.parse_single_example(data, features)

    # FixedLenFeature fields are now ready to use
    sz = tf_record['size']

    # Typical code for decoding compressed images
    image = tf.io.decode_jpeg(tf_record['image'], channels=3)

    # VarLenFeature fields require additional sparse.to_dense decoding
    float_data = tf.sparse.to_dense(tf_record['float_data'])

    return image, sz, float_data

# decoding a tf.data.TFRecordDataset
dataset = dataset.map(read_tfrecord)
# now a dataset of triplets (image, sz, float_data)
```

實用的程式碼片段：

讀取單一資料元素

```
tf.io.FixedLenFeature([], tf.string)    # for one byte string
tf.io.FixedLenFeature([], tf.int64)     # for one int
tf.io.FixedLenFeature([], tf.float32)   # for one float
```

讀取元素固定大小的清單

```
tf.io.FixedLenFeature([N], tf.string)   # list of N byte strings
tf.io.FixedLenFeature([N], tf.int64)    # list of N ints
tf.io.FixedLenFeature([N], tf.float32)  # list of N floats
```

讀取數量不定的資料項目

```
tf.io.VarLenFeature(tf.string)          # list of byte strings
tf.io.VarLenFeature(tf.int64)           # list of ints
tf.io.VarLenFeature(tf.float32)         # list of floats
```

VarLenFeature 會傳回稀疏向量，因此解碼 TFRecord 後還需要額外步驟：

```
dense_data = tf.sparse.to_dense(tf_record['my_var_len_feature'])
```

TFRecord 中也可以有選填欄位。如果在讀取欄位時指定預設值，當欄位遺失時，系統會傳回預設值，而非錯誤。

```
tf.io.FixedLenFeature([], tf.int64, default_value=0) # this field is optional
```

涵蓋內容

- 🤖 分片處理資料檔案，以便從 GCS 快速存取
- 🧐 如何編寫 TFRecords。(您已經忘記語法了嗎？沒關係，請將這個頁面加入書籤，當做快速參考資料)
- 🤖 使用 TFRecordDataset 從 TFRecords 載入資料集

請花點時間在腦中瀏覽這份檢查清單。

6. [INFO] Neural network classifier 101

簡介

如果下一段**粗體**字詞你都瞭解，即可直接進行下一個練習。如果你才剛開始接觸深度學習，歡迎閱讀下文。

類神經網路分類器是由多個**神經元層**組成。如果是圖片分類，這些可以是**密集**或更常見的**捲積層**。通常會使用 **relu** 活化函數啟動。最後一層使用的神經元數量與類別數量相同，並以 **softmax** 啟動。對於分類，**交叉熵**是最常用的損失函式，可比較 **one-hot** 編碼的**標籤** (即正確答案) 與類神經網路預測的機率。為盡量減少損失，最好選擇具有動量的最佳化工具，例如 **AdamOptimizer**，並在訓練圖片和標籤的**批次**上進行訓練。

如要以層序列建構模型，Keras 提供 Sequential API。舉例來說，使用三個密集層的圖片分類器可以 Keras 撰寫，如下所示：

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=[192, 192, 3]),
    tf.keras.layers.Dense(500, activation="relu"),
    tf.keras.layers.Dense(50, activation="relu"),
    tf.keras.layers.Dense(5, activation='softmax') # classifying into 5 classes
])

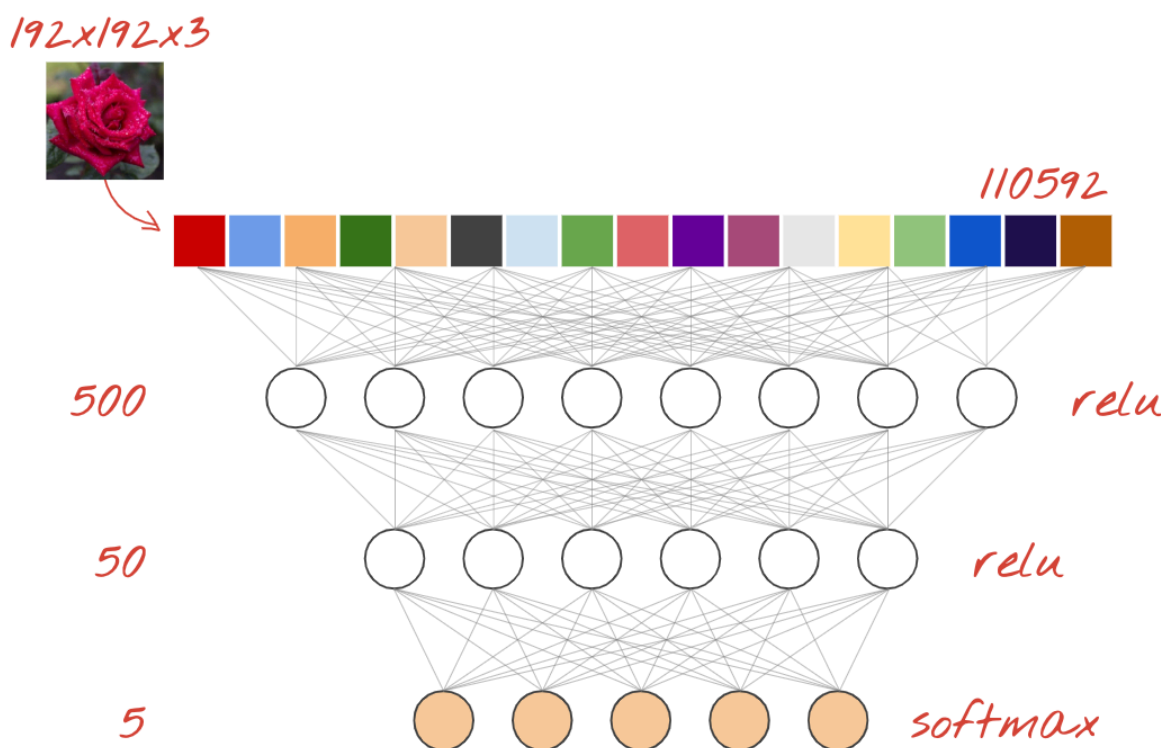
# this configures the training of the model. Keras calls it "compiling" the model.
model.compile(
    optimizer='adam',
    loss= 'categorical_crossentropy',
    metrics=['accuracy']) # % of correct answers

# train the model
model.fit(dataset, ... )
```

If this is old news to you **SKIP**  to the next exercise otherwise **READ ON** 

密集類神經網路

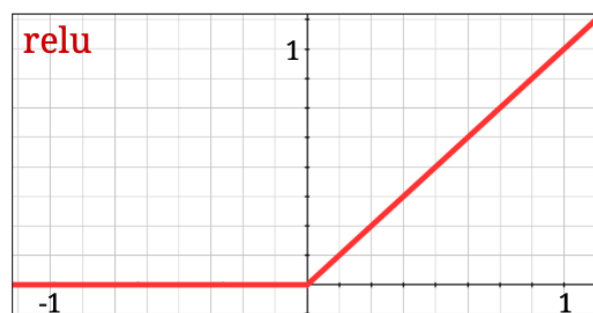
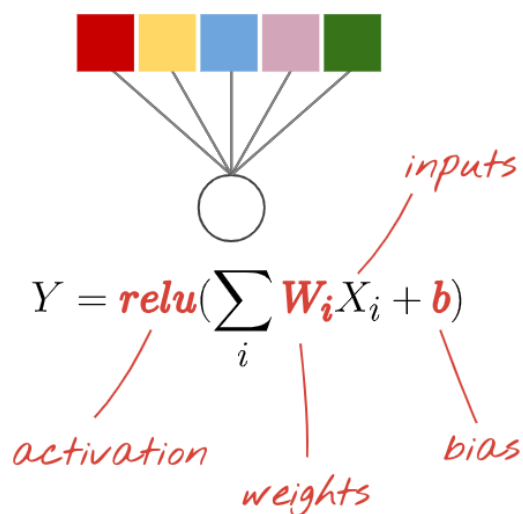
這是最簡單的圖片分類類神經網路，是由分層排列的「神經元」組成。第一層會處理輸入資料，並將輸出內容饋送至其他層。由於每個神經元都會連結至上一層的所有神經元，因此稱為「密集」。



您可以將圖像的所有像素 RGB 值攤平成向量，並做為輸入內容，饋送至這類網路。這並非圖像辨識的最佳技術，但我們之後會加以改良。

神經元、啟動、RELU

「神經元」會計算所有輸入的加權總和，加上稱為「偏差」的值，然後透過所謂的「活化函數」饋送結果。權重和偏差一開始是未知的。這些權重會隨機初始化，並透過大量已知資料訓練類神經網路「學習」。



最常用的活化函數稱為 RELU，也就是線性整流函數。如上圖所示，這項函式非常簡單。

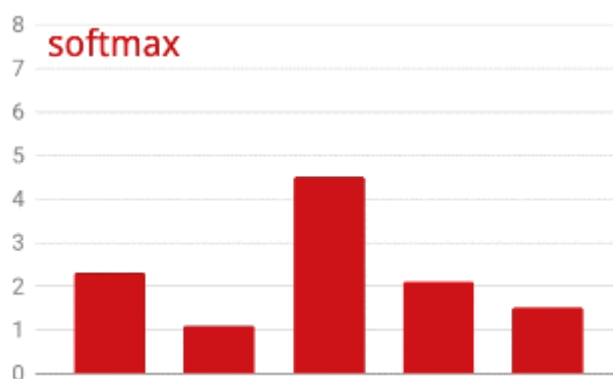
Softmax 啟用

上述網路以 5 個神經元的層級結尾，因為我們要將花朵分類為 5 個類別 (玫瑰、鬱金香、蒲公英、雛菊、向日葵)。中間層的神經元會使用經典的 ReLU 活化函數啟動。不過，在最後一層中，我們想計算介於 0 和 1 之間的數字，代表這朵花是玫瑰、鬱金香等的機率。為此，我們會使用名為「softmax」的活化函數。

對向量套用 Softmax 時，會先計算每個元素的指數，然後將向量正規化，通常是使用 L1 範數 (絕對值總和)，使值加總為 1，並可解讀為機率。

$$\text{softmax}(L_n) = \frac{e^{L_n}}{\|e^L\|}$$

weighted sum+bias *L1 norm*



為什麼「softmax」稱為 softmax？指數是急遽遞增的函式。這會增加神經元輸出之間的差異。接著，當您將向量正規化時，主導常態的最大元素會正規化為接近 1 的值，而所有其他元素則會除以較大的值，並正規化為接近 0 的值。產生的向量會清楚顯示哪個是勝出類別 (即「max」)，但會保留值的原始相對順序，因此是「soft」。

交叉熵損失

現在類神經網路會根據輸入圖片產生預測結果，我們需要測量預測結果的準確度，也就是網路提供的結果與正確答案 (通常稱為「標籤」) 之間的距離。請注意，資料集中的所有圖片都有正確標籤。

任何距離都適用，但對於分類問題，所謂的「交叉熵距離」**最有效**。我們將此稱為誤差或「損失」函式：

1	2	3	4	5
0	0	1	0	0

actual probabilities,
"one-hot" encoded

Cross entropy: $-\sum Y_i' \cdot \log(Y_i)$

computed probabilities

0.2	0.1	0.1	0.5	0.1
-----	-----	-----	-----	-----

1 2 3 4 5
predicted class

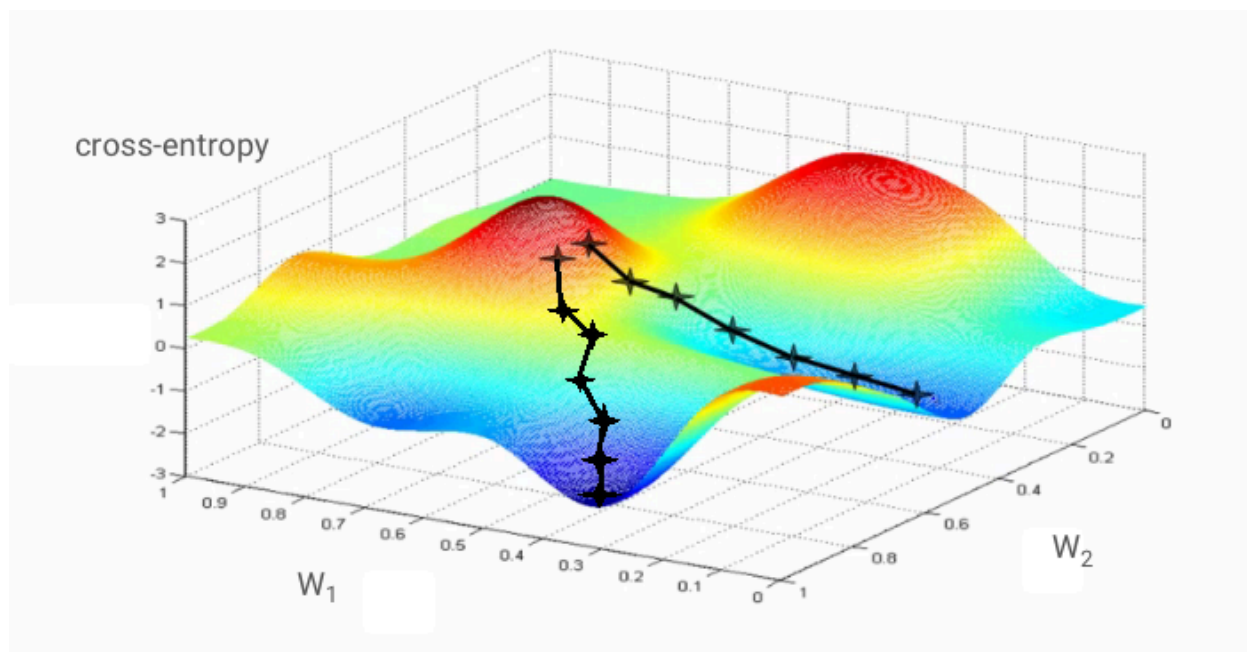
「One-hot」編碼是指使用 5 個值的向量表示「3) 蒲公英」標籤，除了第 3 個值為 1 以外，其餘值皆為零。這個向量代表是蒲公英的機率為 100%。我們的類神經網路也會以 5 個機率值的向量輸出預測結果。方便比較。

梯度下降法

「訓練」類神經網路實際上是指使用訓練圖片和標籤調整權重和偏誤，盡量減少交叉熵損失函式。運作方式如下：

交叉熵是權重、偏差、訓練圖片的像素及其已知類別的函式。

如果我們計算相對於所有權重和所有偏差的交叉熵偏導數，就會得到「梯度」，這是針對特定圖片、標籤，以及權重和偏差的現值計算而得。請注意，我們可能有數百萬個權重和偏差，因此計算梯度聽起來像是大量的工作。幸好，Tensorflow 會為我們執行這項作業。梯度的數學特性是「向上」指向。由於我們希望交叉熵較低，因此要朝相反方向移動。我們會根據梯度的一小部分更新權重和偏差值。接著，我們會在訓練迴圈中，使用下一批訓練圖片和標籤，重複執行相同的操作。希望這會收斂到交叉熵最小的位置，但無法保證這個最小值是唯一的。



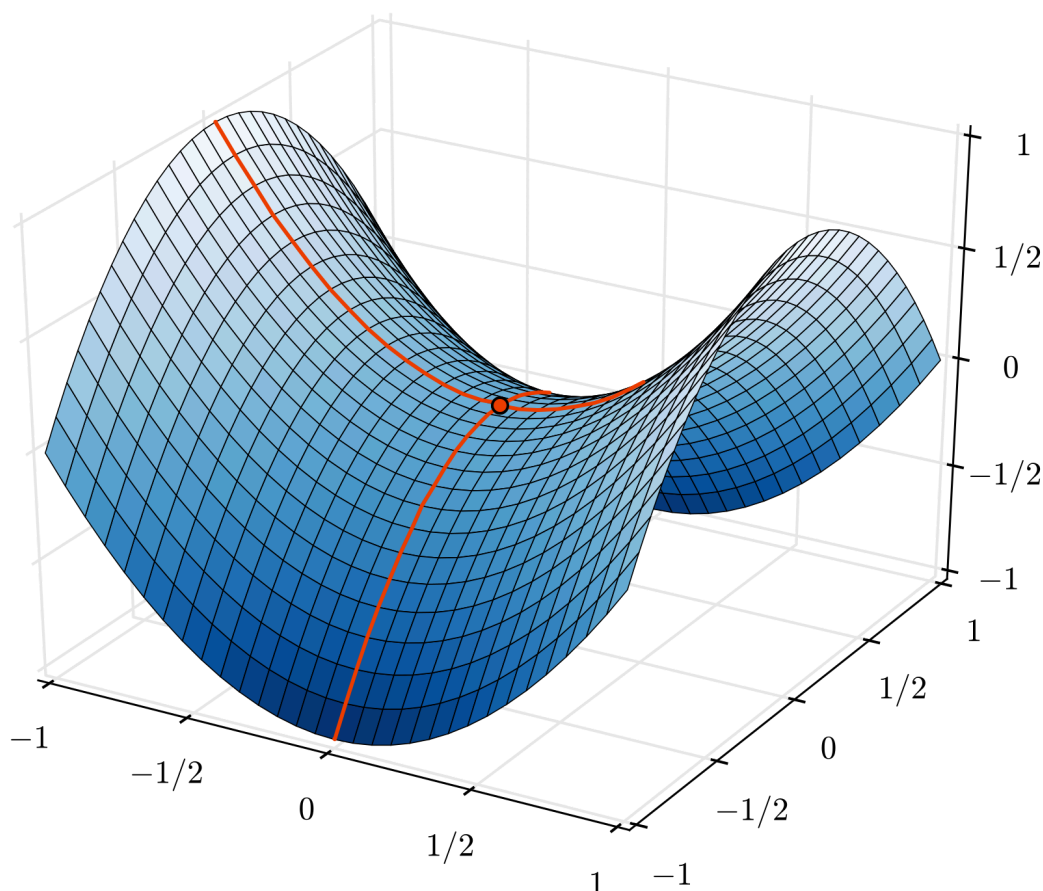
「學習率」：您無法在每次疊代中，依據整個梯度長度更新權重和偏差值。這就像穿著七里靴試圖抵達谷底，您會從山谷的一側跳到另一側。如要到達底部，您必須採取較小的步，也就是只使用一小部分的梯度，通常在 $1/1000$ 範圍內。這個分數稱為「學習率」。

小批次和動量

您可以在單一範例圖片上計算梯度，並立即更新權重和偏差，但如果對一批 (例如 128 張) 圖片執行這項操作，梯度就能更準確地呈現不同範例圖片施加的限制，因此更有可能更快收斂至解決方案。小批次的大小是可調整的參數。

這項技術有時稱為「隨機梯度下降」，還有另一項更實用的優點：處理批次資料也代表要處理較大的矩陣，而這類矩陣通常更容易在 GPU 和 TPU 上進行最佳化。

不過，收斂過程可能還是有點混亂，如果梯度向量全為零，甚至可能會停止。這是否表示我們已找到最小值？不一定。梯度分量在最小值或最大值時可為零。如果梯度向量有數百萬個元素，且全為零，則每個零都對應到最小值，且沒有任何一個對應到最大值的機率相當小。在多維空間中，鞍點相當常見，我們不希望停留在鞍點。



插圖：鞍點。梯度為 0，但並非所有方向的最小值。(圖片出處：[Wikimedia](#)：作者 [Nicoguaro](#) - Own work，CC BY 3.0)

解決方法是為最佳化演算法增加一些動量，讓演算法能順利通過鞍點，不會停滯不前。

Tensorflow 程式庫提供各種最佳化工具，從 `tf.train.GradientDescentOptimizer` 開始。內建動能的進階熱門最佳化工具為 `tf.train.RMSPropOptimizer` 或 `tf.train.AdamOptimizer`。

詞彙

批次或小批：訓練一律會對訓練資料和標籤批次執行。這麼做有助於演算法收斂。「批次」維度通常是資料張量的第一個維度。舉例來說，形狀為 `[100, 192, 192, 3]` 的張量包含 100 張 192x192 像素的圖片，每個像素有三個值 (RGB)。

交叉熵損失：分類器常用的特殊損失函式。

稠密層：神經元層，其中每個神經元都會連結至前一層的所有神經元。

特徵：神經網路的輸入內容有時稱為「特徵」。找出要將資料集中的哪些部分 (或部分組合) 輸入類神經網路，才能獲得準確預測結果，這門技術稱為「特徵工程」。

標籤：監督式分類問題中的「類別」或正確答案的別名

學習率：在訓練迴圈的每次疊代中，權重和偏差值更新的梯度比例。

logits：在套用活化函數之前，神經元層的輸出內容稱為「logits」。這個詞彙源自「邏輯函數」，又稱「S 函數」，這是最常用的活化函數。「Neuron outputs before logistic function」縮短為「logits」。

損失：比較類神經網路輸出內容與正確答案的錯誤函式

神經元：計算輸入內容的加權總和、加入偏差，並透過活化函數提供結果。

One-hot 編碼：5 個類別中的第 3 個類別會編碼為 5 個元素的向量，除了第 3 個元素是 1 以外，其餘都是零。

relu：線性整流函數。神經元的熱門活化函數。

sigmoid：另一種曾廣受歡迎的活化函數，在特殊情況下仍有用處。

softmax：一種特殊活化函數，可作用於向量、增加最大元件與其他元件之間的差異，並將向量正規化為總和為 1，因此可解讀為機率向量。用做分類器的最後一個步驟。

張量：「張量」類似矩陣，但維度數量任意。1 維張量是向量。2 維張量是矩陣。然後，您就可以使用 3、4、5 個以上的維度張量。

7. 遷移學習

如果是圖像分類問題，密集層可能不夠。我們必須瞭解捲積層，以及排列這些層的許多方式。

但我們也可以走捷徑！您可以下載經過完整訓練的卷積類神經網路。您可以將最後一層 (softmax 分類標頭) 截斷，並換成自己的標頭。所有訓練權重和偏差都會維持不變，您只會重新訓練新增的 Softmax 層。這項技術稱為「遷移學習」，只要預先訓練類神經網路的資料集與您的資料集「夠接近」，這項技術就能發揮作用。

實作

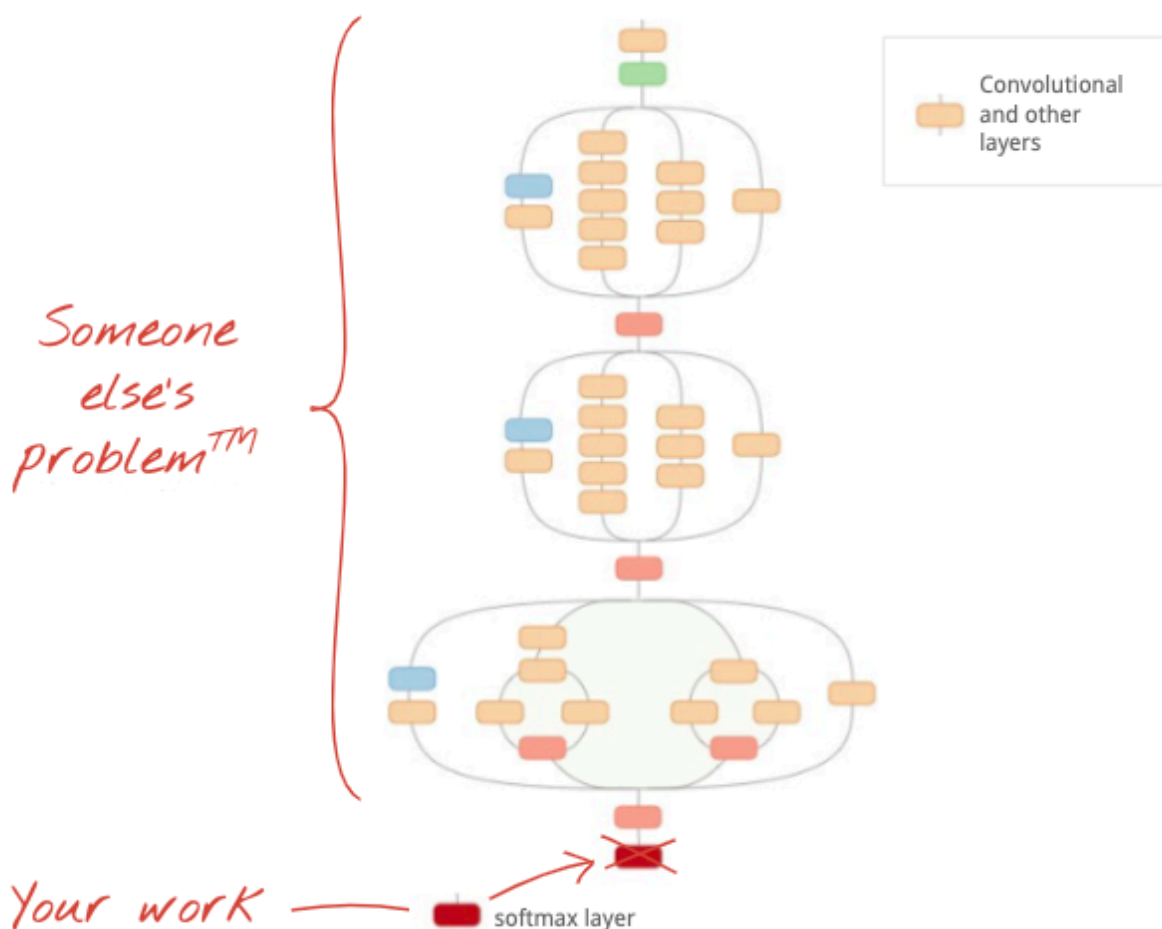
請開啟下列筆記本、執行儲存格 (Shift-ENTER)，並按照標示「WORK REQUIRED」的指示操作。



[Keras Flowers transfer learning \(playground\).ipynb](#)

其他資訊

透過遷移學習，您可同時享有頂尖研究人員開發的先進卷積類神經網路架構，以及在龐大圖片資料集上預先訓練的優勢。在本例中，我們將從 ImageNet 訓練的網路進行遷移學習。ImageNet 是包含許多植物和戶外場景的圖片資料庫，與花卉的相似度相當高。



插圖：使用已訓練的複雜卷積類神經網路做為黑箱，僅重新訓練分類標頭。這就是遷移學習。我們稍後會瞭解這些複雜的捲積層排列方式如何運作。目前這是其他人的問題。

Keras 中的遷移學習

在 Keras 中，您可以從 `tf.keras.applications.*` 集合例項化預先訓練模型。舉例來說，MobileNet V2 是非常出色的卷積架構，大小合理。選取 `include_top=False` 即可取得預先訓練模型，但沒有最終的 Softmax 層，因此您可以自行新增：

```
pretrained_model = tf.keras.applications.MobileNetV2(input_shape=[*IMAGE_SIZE, 3],
include_top=False)
pretrained_model.trainable = False

model = tf.keras.Sequential([
    pretrained_model,
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(5, activation='softmax')
])
```

另請注意 `pretrained_model.trainable = False` 設定。這會凍結預先訓練模型的權重和偏誤，因此您只會訓練 Softmax 層。這通常只涉及相對較少的權重，而且可以快速完成，不需要非常龐大的資料集。不過，如果您有大量資料，遷移學習搭配 `pretrained_model.trainable = True` 的效果會更好。預先訓練的權重可提供絕佳的初始值，且仍可透過訓練調整，以更符合您的問題。

最後，請注意插入密集 softmax 層之前的 `Flatten()` 層。密集層可處理資料的平坦向量，但我們不知道預先訓練的模型是否會傳回這類向量。因此我們需要將其攤平。在下一章深入探討卷積架構時，我們會說明卷積層傳回的資料格式。

使用這種方法，準確率應可接近 75%。

解決方案

解決方案筆記本在此。如果遇到困難，可以參考這項工具。



[Keras Flowers transfer learning \(solution\).ipynb](#)

涵蓋內容

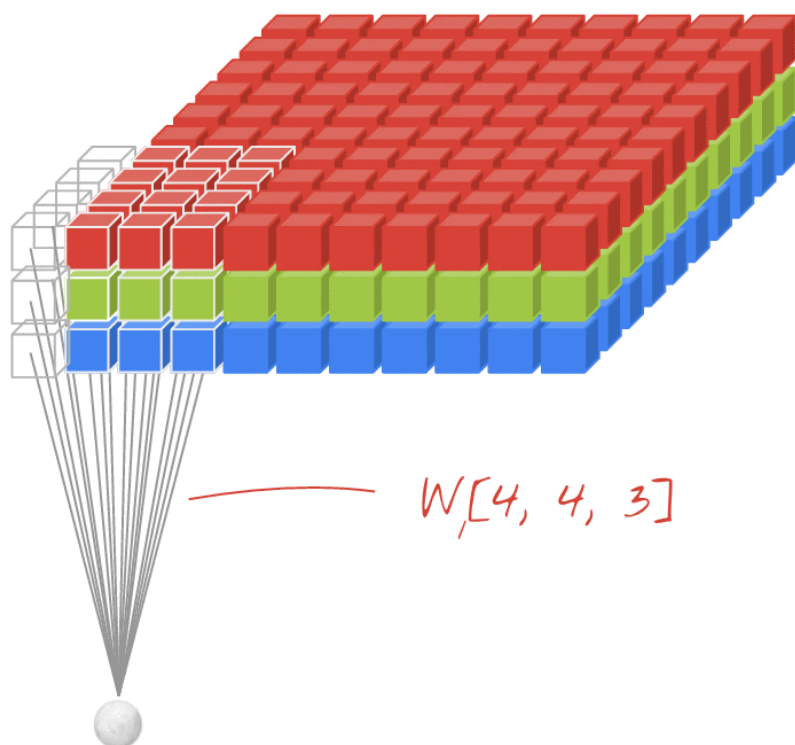
- 🤖 如何在 Keras 中編寫分類器
- 🧠，並設定 softmax 最後一層和交叉熵損失
- 🐙 遷移學習
- 🧑‍🔬 訓練第一個模型
- 📊 追蹤訓練期間的損失和準確率

請花點時間在腦中瀏覽這份檢查清單。

8. [INFO] 卷積類神經網路

簡介

如果下一段**粗體**字詞你都瞭解，即可直接進行下一個練習。如果您才剛開始接觸卷積類神經網路，請繼續閱讀。



插圖：使用兩個連續的濾鏡篩選圖片，每個濾鏡由 $4 \times 4 \times 3 = 48$ 個可學習權重組成。

卷積類神經網路會對輸入圖片套用一系列可學習的濾鏡。卷積層是由濾鏡 (或核心) 大小、套用的濾鏡數量和步幅所定義。捲積層的輸入和輸出都有三個維度 (寬度、高度、通道數)，從輸入圖片 (寬度、高度、RGB 通道) 開始。堆疊卷積層時，可以使用跨距 >1 或 **max-pooling** 運算，調整輸出內容的寬度和高度。使用更多或更少的篩選器，即可調整輸出內容的深度 (通道數)。

以下是 Keras 中簡單的卷積類神經網路：

```

model = tf.keras.Sequential([
    # input: images of size 192x192x3 pixels (the three stands for RGB channels)
    tf.keras.layers.Conv2D(kernel_size=3, filters=24, padding='same', activation='relu',
input_shape=[192, 192, 3]),
    tf.keras.layers.Conv2D(kernel_size=3, filters=24, padding='same', activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=2),
    tf.keras.layers.Conv2D(kernel_size=3, filters=12, padding='same', activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=2),
    tf.keras.layers.Conv2D(kernel_size=3, filters=6, padding='same', activation='relu'),
    tf.keras.layers.Flatten(),
    # classifying into 5 categories
    tf.keras.layers.Dense(5, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss= 'categorical_crossentropy',
    metrics=['accuracy'])

```

If this is old news to you **SKIP**  to the next exercise otherwise **READ ON** 

卷積類神經網路 101

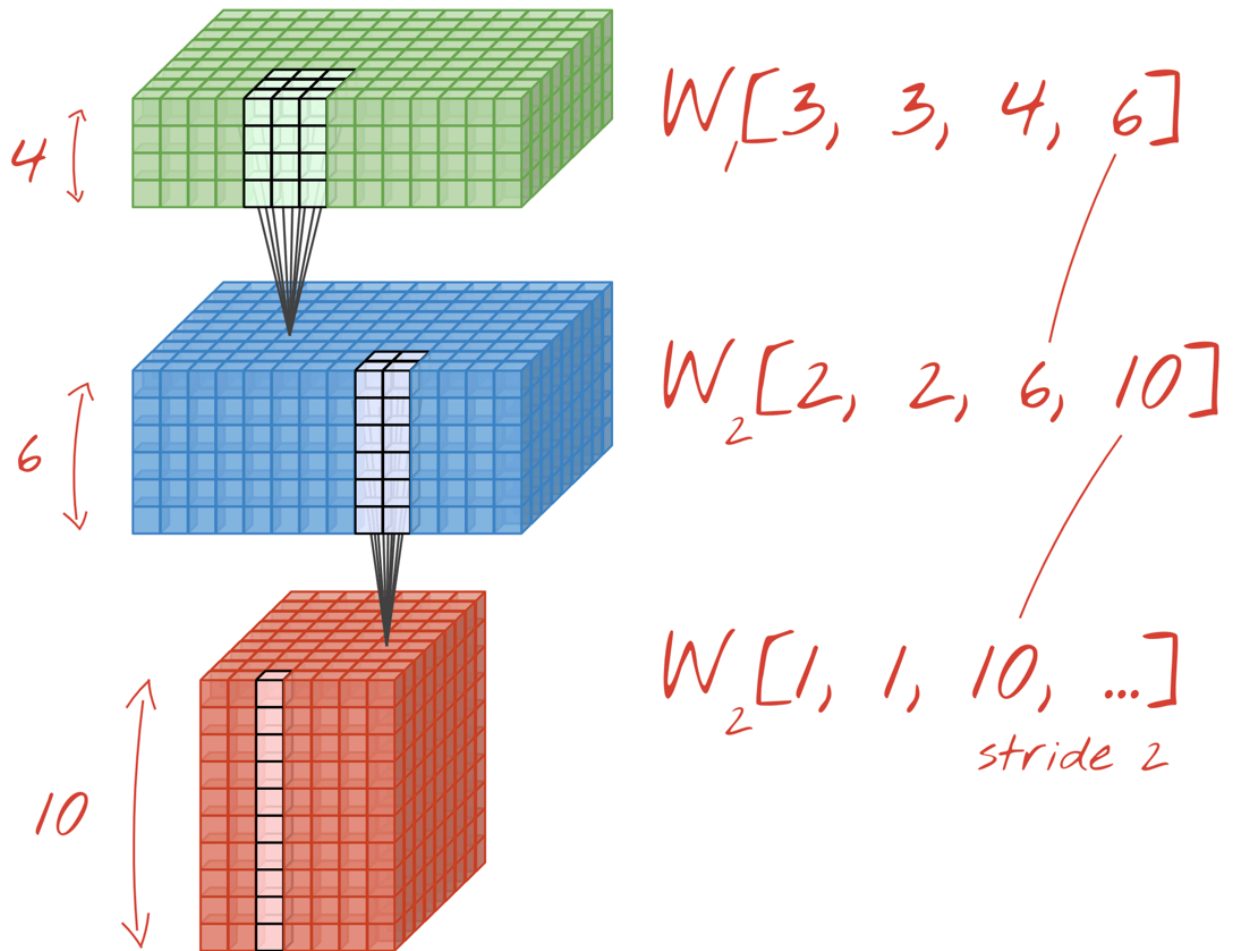
在卷積網路的層中，一個「神經元」只會對方一小塊圖像區域的像素進行加權總和。它會新增偏差值，並透過活化函數饋送總和，就像一般稠密層中的神經元一樣。然後使用相同權重，在整張圖片上重複執行這項作業。請注意，在密集層中，每個神經元都有自己的權重。在此，單一「權重修補程式」會以兩個方向滑過圖片 (即「捲積」)。輸出值數量與圖片中的像素數量相同 (但邊緣需要一些邊框間距)。這是篩選作業，使用 $4 \times 4 \times 3 = 48$ 個權重的篩選器。

不過，48 個權重並不夠。如要增加自由度，請使用一組新的權重重複相同作業。系統會產生一組新的篩選器輸出內容。以輸入圖片中的 R、G、B 通道類比，我們將輸出內容稱為「通道」。

$$\begin{array}{c}
 W_1[4, 4, 3] \\
 W_2[4, 4, 3]
 \end{array}
 \Bigg|
 W[4, 4, 3, 2]$$

filter size
input channels
output channels

這兩組 (或更多組) 權重可以透過新增維度加總為一個張量。這會提供卷積層權重張量的通用形狀。由於輸入和輸出通道的數量是參數，我們可以開始堆疊和串連捲積層。

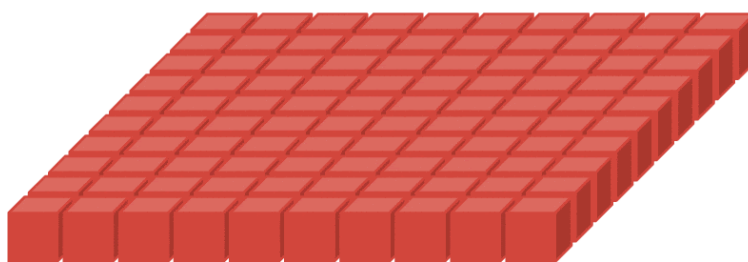


插圖：卷積類神經網路將資料「立方體」轉換為其他資料「立方體」。

步進卷積、最大池化

以 2 或 3 的步幅執行捲積時，我們也可以縮減結果資料方塊的水平維度。常見做法有兩種：

- 步進式卷積：滑動篩選器，但跨距 > 1
- 最大集區化：套用 MAX 作業的滑動視窗 (通常在 2x2 修補程式上，每 2 像素重複一次)



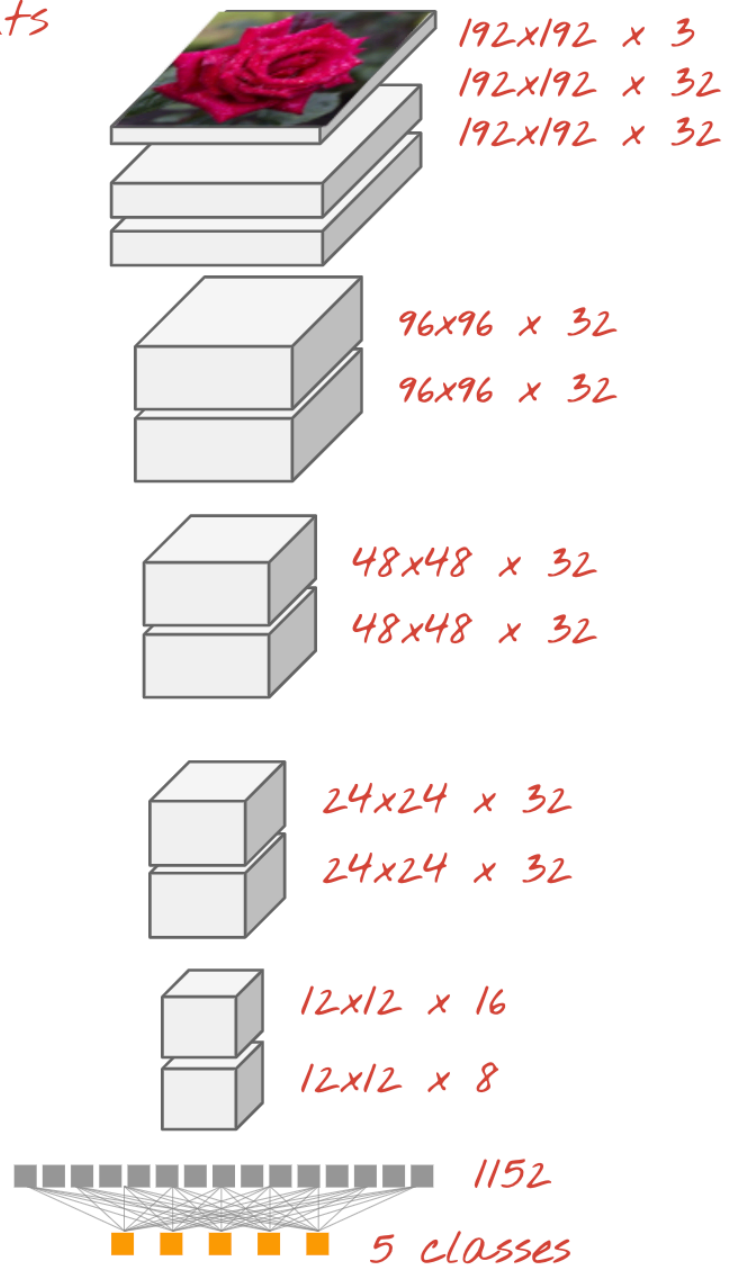
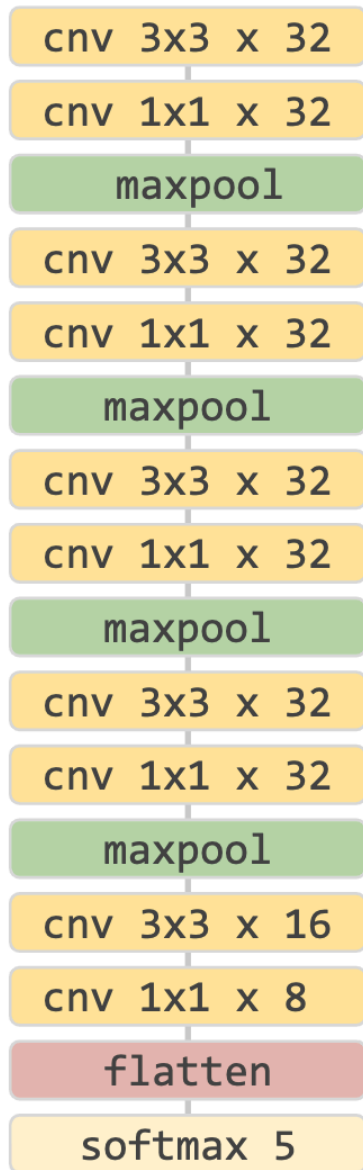
插圖：將運算視窗滑動 3 像素，輸出值就會減少。步幅式捲積或最大集區化 (在步幅為 2 的 2x2 視窗上取最大值) 是縮減水平維度資料立方體的方法。

「最大池化」可協助我們直覺地瞭解卷積網路的運作方式：假設在訓練期間，權重的修補程式會演變成可辨識基本形狀 (水平和垂直線、曲線等) 的篩選器，那麼簡化實用資訊的方法之一，就是保留各層中辨識出形狀且強度最大的輸出內容。實務上，在最大集區層中，神經元輸出會以 2x2 的群組處理，且只會保留最大值。

Convolutional classifier

最後，我們將最後一個資料立方體攤平，並透過密集、softmax 啟用的層饋送，附加分類標題。典型的捲積分類器可能如下所示：

11 layers 8K weights



插圖：使用捲積和 Softmax 層的图片分類器。它使用 3x3 和 1x1 濾鏡。maxpool 層會取 2x2 資料點群組的最大值。分類標頭是透過具有 Softmax 啟動的稠密層實作。

在 Keras 中

上述的捲積堆疊可以寫成 Keras 程式碼，如下所示：

```

model = tf.keras.Sequential([
    # input: images of size 192x192x3 pixels (the three stands for RGB channels)
    tf.keras.layers.Conv2D(kernel_size=3, filters=32, padding='same', activation='relu',
input_shape=[192, 192, 3]),
    tf.keras.layers.Conv2D(kernel_size=1, filters=32, padding='same', activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=2),
    tf.keras.layers.Conv2D(kernel_size=3, filters=32, padding='same', activation='relu'),
    tf.keras.layers.Conv2D(kernel_size=1, filters=32, padding='same', activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=2),
    tf.keras.layers.Conv2D(kernel_size=3, filters=32, padding='same', activation='relu'),
    tf.keras.layers.Conv2D(kernel_size=1, filters=32, padding='same', activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=2),
    tf.keras.layers.Conv2D(kernel_size=3, filters=32, padding='same', activation='relu'),
    tf.keras.layers.Conv2D(kernel_size=1, filters=32, padding='same', activation='relu'),
    tf.keras.layers.MaxPooling2D(pool_size=2),
    tf.keras.layers.Conv2D(kernel_size=3, filters=16, padding='same', activation='relu'),
    tf.keras.layers.Conv2D(kernel_size=1, filters=8, padding='same', activation='relu'),
    tf.keras.layers.Flatten(),
    # classifying into 5 categories
    tf.keras.layers.Dense(5, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss= 'categorical_crossentropy',
    metrics=[ 'accuracy' ])

```

卷積層的邊框間距參數可有兩個值：

- 「same」：以零填補，產生與輸入相同寬度/高度的輸出
- 「valid」：無邊框間距，僅使用實際像素

9. 您的自訂轉換網路

實作

讓我們從頭開始建構及訓練卷積類神經網路。使用 TPU 可讓我們快速疊代。請開啟下列筆記本、執行儲存格 (Shift-ENTER)，並按照標示「WORK REQUIRED」的指示操作。



[Keras_Flowers_TPU \(playground\).ipynb](#)

目標是超越遷移學習模型 75% 的準確度。該模型已預先在數百萬張圖片的資料集上訓練，而我們這裡只有 3670 張圖片，因此該模型具有優勢。至少可以比照辦理嗎？

其他資訊

有幾層、多大？

選取圖層大小是藝術，而非科學。您必須在參數 (權重和偏差) 過少和過多之間取得適當平衡。如果權重太少，類神經網路就無法呈現花朵形狀的複雜性。如果特徵過多，模型就容易「過度配適」，也就是專門處理訓練圖片，但無法一般化。如果參數過多，模型訓練速度也會變慢。在 Keras 中，`model.summary()` 函式會顯示模型的結構和參數計數：

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 192, 192, 16)	448
conv2d_1 (Conv2D)	(None, 192, 192, 30)	4350
max_pooling2d (MaxPooling2D)	(None, 96, 96, 30)	0
conv2d_2 (Conv2D)	(None, 96, 96, 60)	16260
...		
global_average_pooling2d (Gl	(None, 130)	0
dense (Dense)	(None, 90)	11790
dense_1 (Dense)	(None, 5)	455
Total params: 300,033		
Trainable params: 300,033		
Non-trainable params: 0		

以下提供幾項訣竅：

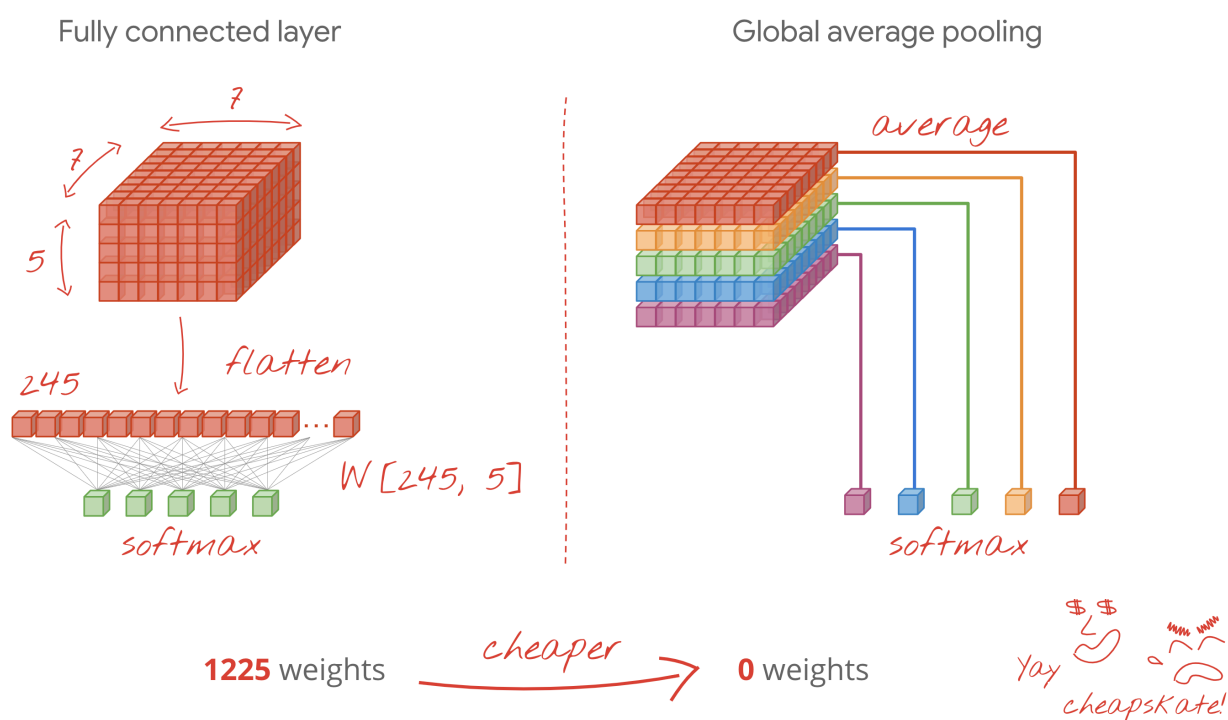
- 多層結構是深層類神經網路發揮效用的關鍵。以這個簡單的花朵辨識問題為例，5 到 10 層就足夠。
- 使用小型篩選器。一般來說，3x3 濾鏡在任何地方都適用。
- 您也可以使用 1x1 濾鏡，價格便宜。它們並不會「篩選」任何內容，而是計算通道的線性組合。並以實際篩選器替換。(下一節會進一步說明「1x1 捲積」)。

- 對於這類分類問題，請使用最大集區層 (或步幅 >1 的捲積) 頻繁地進行下採樣。您不在意花朵的位置，只在意花朵是玫瑰還是蒲公英，因此遺失 x 和 y 資訊並不重要，且篩選較小的區域成本較低。
- 網路結尾的篩選器數量通常會與類別數量相近 (原因請見下方的「全域平均集區化」訣竅)。如果分類數達到數百個，請在連續層中逐步增加篩選器數量。以含有 5 個類別的花朵資料集為例，只使用 5 個篩選器是不夠的。您可以在大多數層中使用相同的篩選器數量 (例如 32 個)，並在結尾減少篩選器數量。
- 最後的稠密層成本高昂。這類層的權重可能比所有捲積層加總的權重還多。舉例來說，即使最後一個資料立方體的輸出非常合理，只有 $24 \times 24 \times 10$ 個資料點，100 個神經元的稠密層仍會耗費 $24 \times 24 \times 10 \times 100 = 576,000$ 個權重！請謹慎思考，或嘗試全域平均集區化 (見下文)。

全域平均池化

您不必在卷積類神經網路的結尾使用昂貴的稠密層，而是可以將傳入的資料「立方體」分割成與類別數量相同的多個部分、計算這些部分的平均值，然後透過 Softmax 活化函數饋送這些值。這種建構分類標題的方式不會產生權重費用。在

Keras 中，語法為 `tf.keras.layers.GlobalAveragePooling2D()`。



卷積類神經網路會偵測物體的位置。當濾鏡對某些特徵有強烈反應時，會在特定 x、y 位置產生反應。視用途而定，類神經網路可以訓練成使用或捨棄這類位置資料。明確使用全域平均集區化會捨棄所有位置資料。這對花朵分類器來說可能沒問題，但對其他工作 (例如計算圖片中的花朵數量) 來說可能就不適用。請謹慎使用。

解決方案

解決方案筆記本在此。如果遇到困難，可以參考這項工具。



[Keras_Flowers_TPU \(solution\).ipynb](#)

涵蓋內容

- 🤖 玩過卷積層

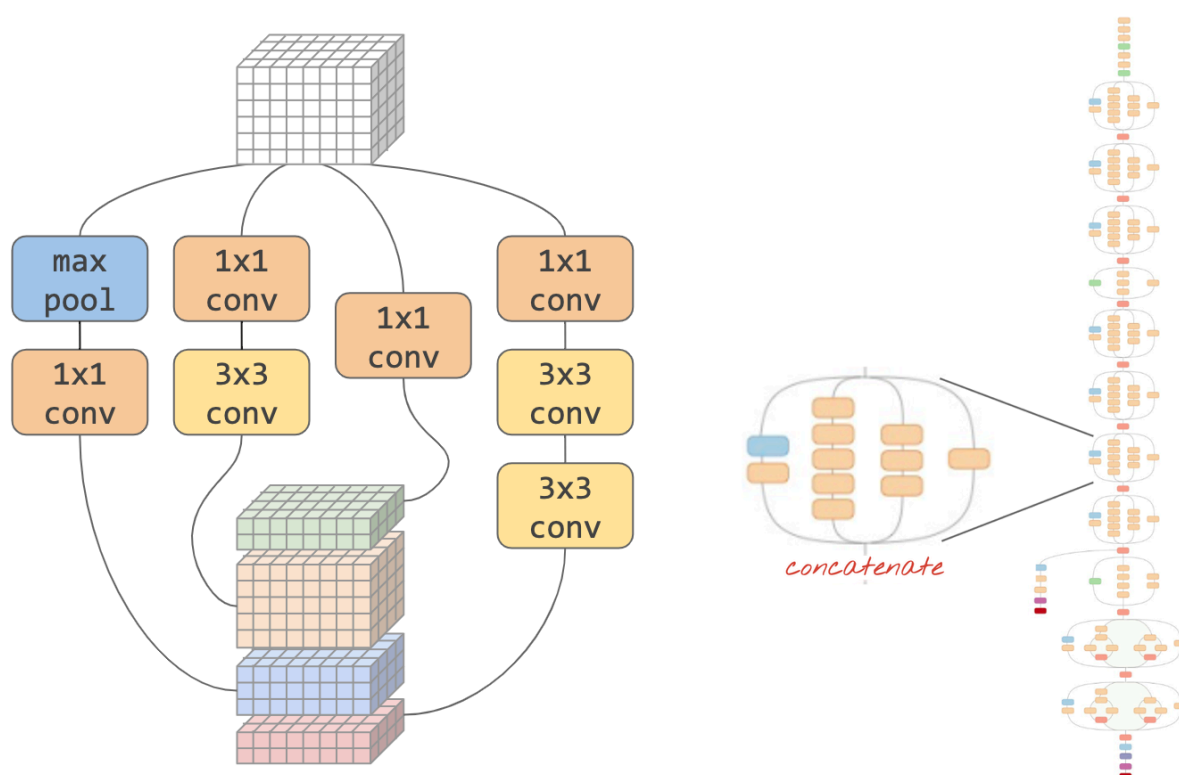
- 🤖 實驗過最大集區化、步幅、全域平均集區化等。
- 😊 在 TPU 上快速疊代實際模型

請花點時間在腦中瀏覽這份檢查清單。

10. [INFO] 現代卷積架構

簡介

以「[Inception](#)」和「[Inception v2](#)」為首的現代捲積網路，通常會使用「模組」，在同一個輸入內容上同時嘗試不同的捲積層，將輸出內容串連起來，並透過訓練決定哪個層最有用。



插圖：卷積「模組」。此時最適合做什麼？最大池化層，後面接著 1x1 卷積層，或是其他層的組合？請嘗試所有方法、串連結果，然後讓網路決定。右側：使用這類模組的「[inception](#)」卷積架構。

在 Keras 中，如要建立資料流程可分支進出的模型，必須使用「函式」模型樣式。範例如下：

```

l = tf.keras.layers # syntax shortcut

y = l.Conv2D(filters=32, kernel_size=3, padding='same',
             activation='relu', input_shape=[192, 192, 3])(x) # x=input image

# module start: branch out
y1 = l.Conv2D(filters=32, kernel_size=1, padding='same', activation='relu')(y)
y3 = l.Conv2D(filters=32, kernel_size=3, padding='same', activation='relu')(y)
y = l.concatenate([y1, y3]) # output now has 64 channels
# module end: concatenation

# many more layers ...

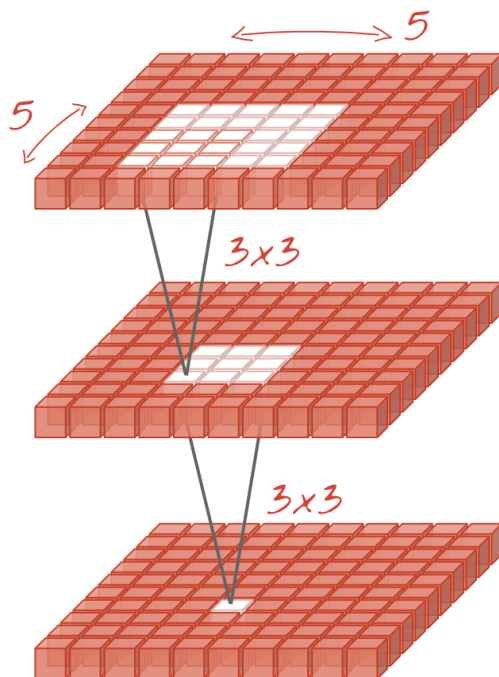
# Create the model by specifying the input and output tensors.
# Keras layers track their connections automatically so that's all that's needed.
z = l.Dense(5, activation='softmax')(y)
model = tf.keras.Model(x, z)

```

If this is old news to you **SKIP**  to the next exercise otherwise **READ ON** 

其他廉價手法

小型 3x3 濾鏡



5x5 filter $W[5,5,1,1] = 25$ weights

3x3 filter $W_1[3,3,1,1]$

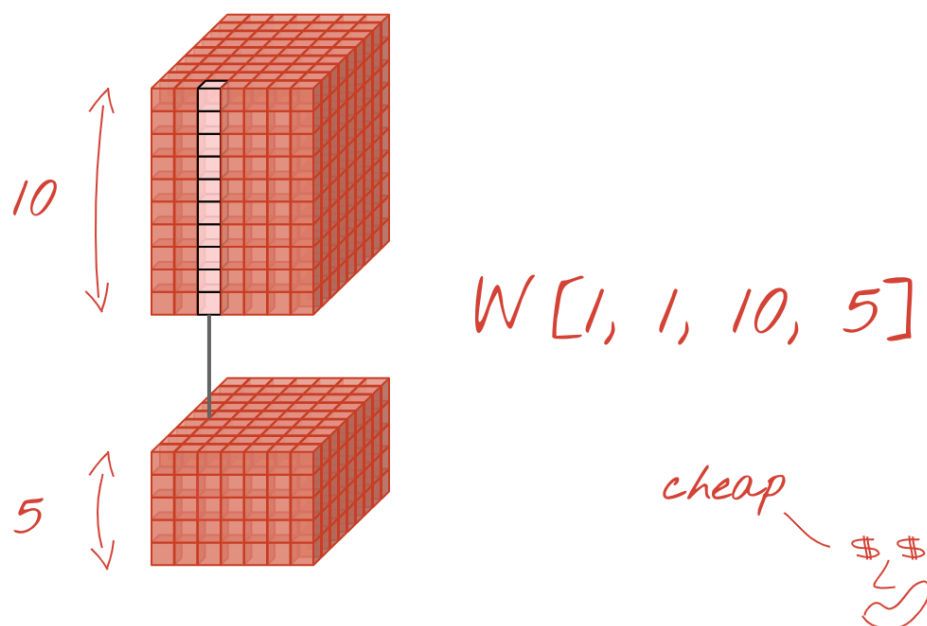
3x3 filter $W_2[3,3,1,1] = 9+9=18$ weights

30%
cheaper



這張插圖顯示兩個連續 3x3 濾鏡的結果。請嘗試追蹤哪些資料點促成了結果：這兩個連續的 3x3 篩選器會計算 5x5 區域的某種組合。這與 5x5 篩選器運算的組合並不完全相同，但值得嘗試，因為兩個連續的 3x3 篩選器比單一 5x5 篩選器便宜。

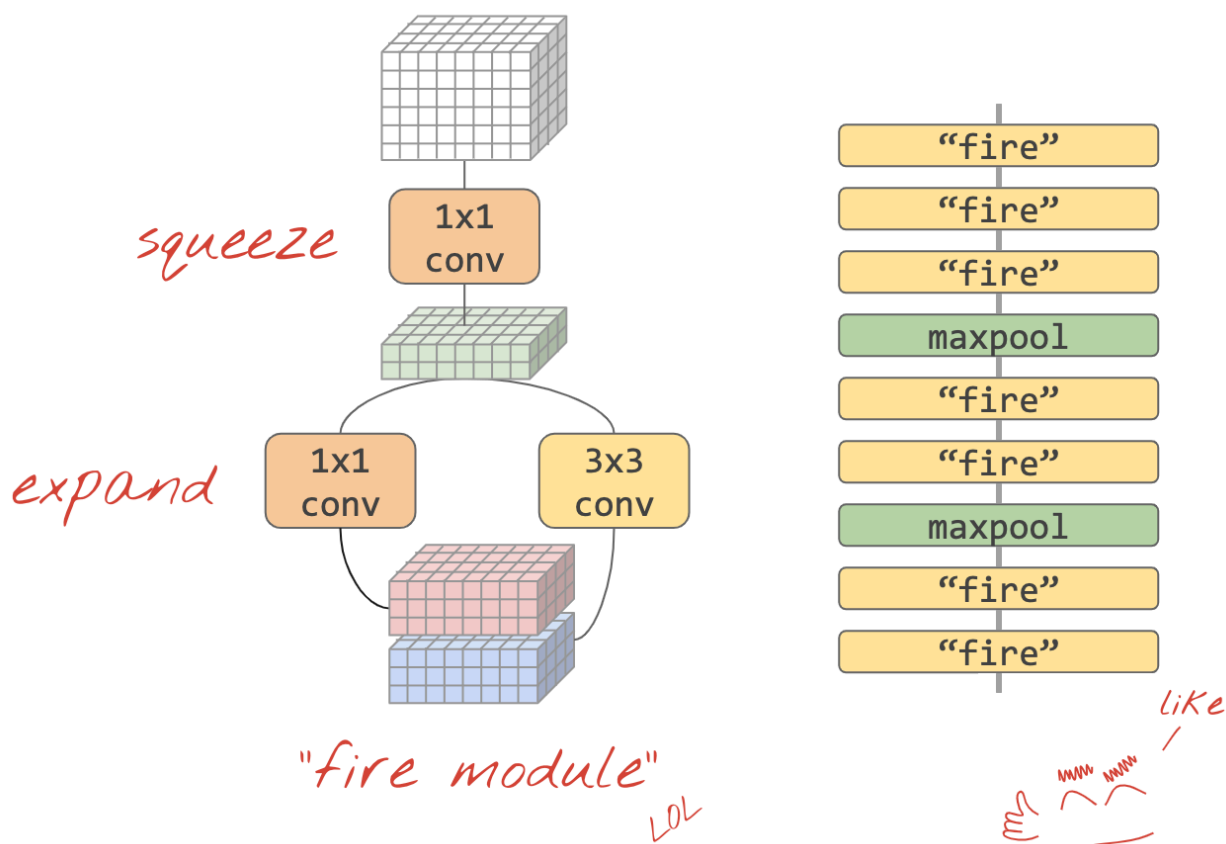
1x1 卷積？



以數學術語來說，「1x1」的捲積是乘以常數，並非很有用的概念。不過請注意，在卷積神經網路中，篩選器會套用至資料立方體，而不只是 2D 圖片。因此，「1x1」篩選器會計算 1x1 資料欄的加權總和 (請參閱插圖)，並在您滑動資料時，取得輸入管道的線性組合。這其實很有用。如果將管道視為個別篩選作業的結果，例如「尖耳朵」的篩選器、「鬍鬚」的篩選器，以及「細縫眼睛」的篩選器，那麼「1x1」卷積層就會計算這些特徵的多種可能線性組合，這在尋找「貓」時可能很有用。此外，1x1 層使用的權重較少。

11. Squeezenet

「Squeezenet」[論文](#)中展示了將這些概念整合在一起的簡單方法。作者建議採用非常簡單的捲積模組設計，僅使用 1x1 和 3x3 捲積層。



插圖：以「Fire 模組」為基礎的 SqueezeNet 架構。這些層會交替使用 1x1 層，先「擠壓」垂直維度中的輸入資料，再使用兩個平行的 1x1 和 3x3 卷積層，再次「擴展」資料深度。

實作

繼續使用先前的筆記本，並建構以 SqueezeNet 為靈感的卷積類神經網路。您必須將模型程式碼變更為 Keras 「函式樣式」。



[Keras_Flowers_TPU \(playground\).ipynb](#)

其他資訊

請將這些訓練的訓練週期計數調高至 35。

定義 squeezenet 模組的輔助函式，對這項練習很有幫助：

```
def fire(x, squeeze, expand):
    y = l.Conv2D(filters=squeeze, kernel_size=1, padding='same', activation='relu')(x)
    y1 = l.Conv2D(filters=expand//2, kernel_size=1, padding='same', activation='relu')(y)
    y3 = l.Conv2D(filters=expand//2, kernel_size=3, padding='same', activation='relu')(y)
    return tf.keras.layers.concatenate([y1, y3])

# this is to make it behave similarly to other Keras layers
def fire_module(squeeze, expand):
    return lambda x: fire(x, squeeze, expand)

# usage:
x = l.Input(shape=[192, 192, 3])
y = fire_module(squeeze=24, expand=48)(x) # typically, squeeze is less than expand
y = fire_module(squeeze=32, expand=64)(y)
...
model = tf.keras.Model(x, y)
```

這次的目標是達到 80% 的準確率。

建議做法

先從單一卷積層開始，然後依序加入「fire_modules」，並與 MaxPooling2D(pool_size=2) 層交替。您可以在網路中實驗 2 到 4 個最大集區化層，也可以在最大集區化層之間實驗 1、2 或 3 個連續的 Fire 模組。

在 fire 模組中，「squeeze」參數通常應小於「expand」參數。這些參數實際上是篩選器的數量。通常介於 8 到 196 之間。您可以實驗架構，讓網路中的篩選器數量逐漸增加，也可以使用簡單的架構，讓所有 Fire 模組的篩選器數量相同。

範例如下：

```
x = tf.keras.layers.Input(shape=[*IMAGE_SIZE, 3]) # input is 192x192 pixels RGB

y = tf.keras.layers.Conv2D(kernel_size=3, filters=32, padding='same', activation='relu')(x)
y = fire_module(24, 48)(y)
y = tf.keras.layers.MaxPooling2D(pool_size=2)(y)
y = fire_module(24, 48)(y)
y = tf.keras.layers.MaxPooling2D(pool_size=2)(y)
y = fire_module(24, 48)(y)
y = tf.keras.layers.GlobalAveragePooling2D()(y)
y = tf.keras.layers.Dense(5, activation='softmax')(y)

model = tf.keras.Model(x, y)
```

此時您可能會發現實驗成效不佳，80% 的準確率目標似乎遙不可及。現在要介紹幾個更簡單的技巧。

批次正規化

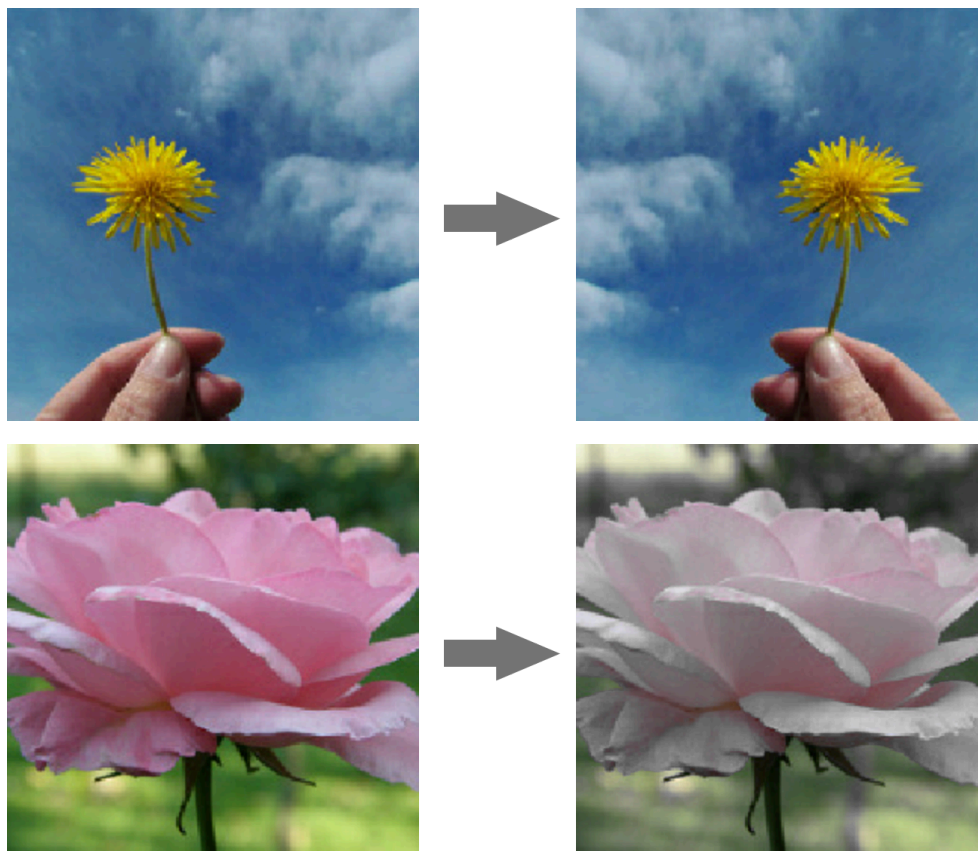
批次正規化有助於解決您遇到的收斂問題。我們會在下一個研討會中詳細說明這項技術，目前請先將其視為「神奇」的黑箱輔助工具，在網路中每個卷積層之後加入這行程式碼，包括 fire_module 函式內的層：

```
y = tf.keras.layers.BatchNormalization(momentum=0.9)(y)
# please adapt the input and output "y"s to whatever is appropriate in your context
```


由於資料集較小，動量參數必須從預設值 0.99 減少至 0.9。目前先不用理會這項細節。

資料擴增

透過簡單的轉換 (例如飽和度變化的左右翻轉) 擴增資料，即可獲得更多百分比點數：



在 TensorFlow 中，使用 `tf.data.Dataset` API 即可輕鬆完成這項作業。為資料定義新的轉換函式：

```
def data_augment(image, label):  
    image = tf.image.random_flip_left_right(image)  
    image = tf.image.random_saturation(image, lower=0, upper=2)  
    return image, label
```

然後在最終資料轉換中使用 (「訓練和驗證資料集」儲存格，函式「`get_batched_dataset`」)：

```
dataset = dataset.repeat() # existing line  
# insert this  
if augment_data:  
    dataset = dataset.map(data_augment, num_parallel_calls=AUTO)  
dataset = dataset.shuffle(2048) # existing line
```

別忘了將資料擴增設為選用，並新增必要程式碼，確保只有訓練資料集會擴增。擴增驗證資料集沒有意義。

現在應該可以在 35 個訓練週期內達到 80% 的準確率。

解決方案

解決方案筆記本在此。如果遇到困難，可以參考這項工具。



`Keras_Flowers_TPU_squeezenet.ipynb`

涵蓋內容

- 🤖 Keras 「函式樣式」 模型
- 🤖 Squeezenet 架構
- 🤖 使用 `tf.data.dataset` 擴增資料

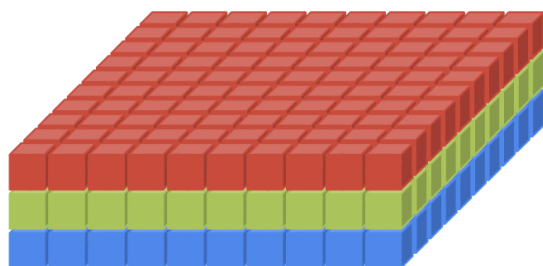
請花點時間在腦中瀏覽這份檢查清單。

12. 微調 Xception

可分離的捲積

最近，另一種實作卷積層的方式越來越受歡迎，那就是深度可分離卷積。我知道這有點難記，但概念相當簡單。這些層級在 TensorFlow 和 Keras 中會實作為 `tf.keras.layers.SeparableConv2D`。

可分離卷積也會對圖片執行濾波器，但會為輸入圖片的每個管道使用不同的權重集。接著是「1x1 捲積」，也就是一系列的點積，可得出經過濾波處理的通道加權總和。每次都會使用新的權重，並視需要計算多個管道的加權重組。



插圖：可分離的捲積。第 1 階段：對每個聲道使用個別的濾鏡進行捲積。第 2 階段：管道的線性重組。使用一組新的權重重複上述步驟，直到達到所需的輸出通道數。階段 1 也可以重複執行，每次都使用新的權重，但實際上很少這麼做。

在最新的卷積網路架構中，大多會使用可分離的卷積，例如 MobileNetV2、Xception 和 EfficientNet。順帶一提，MobileNetV2 是您先前用於遷移學習的項目。

這類卷積比一般卷積便宜，且實務上效果相同。以下是上述範例的權重計數：

卷積層： $4 \times 4 \times 3 \times 5 = 240$

可分離卷積層： $4 \times 4 \times 3 + 3 \times 5 = 48 + 15 = 63$

讀者可自行計算套用每種捲積層級時所需的乘法運算次數。可分離式捲積較小，運算效率也高得多。

實作

從「遷移學習」Playground 筆記本重新開始，但這次請選取 Xception 做為預先訓練模型。Xception 只使用可分離的捲積。讓所有權重都能訓練。我們會根據資料微調預先訓練的權重，而不是直接使用預先訓練的層。



Keras Flowers transfer learning (playground).ipynb

目標：準確度高於 95% (真的，這有可能！)

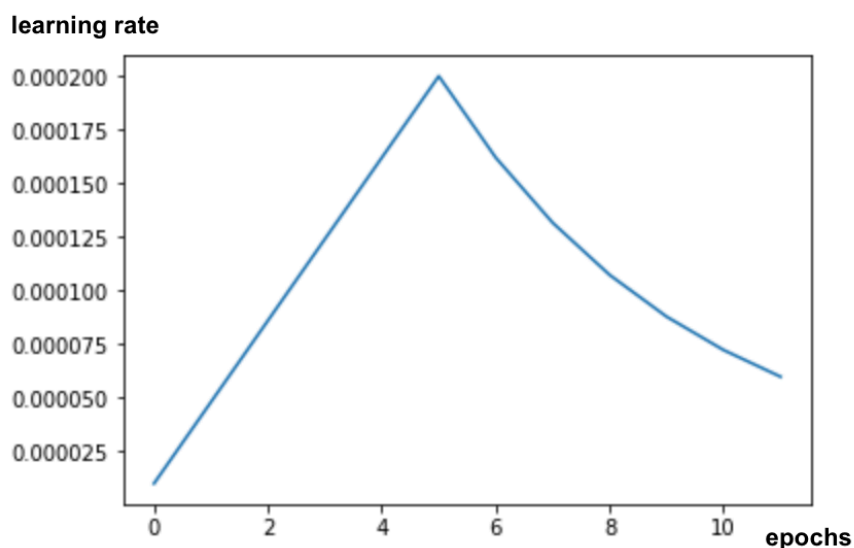
這是最後一個練習，因此需要更多程式碼和資料科學工作。

微調的其他資訊

`tf.keras.application.*` 中的標準預先訓練模型提供 Xception，請勿忘記這次要將所有權重設為可訓練。

```
pretrained_model = tf.keras.applications.Xception(input_shape=[*IMAGE_SIZE, 3],
                                                  include_top=False)
pretrained_model.trainable = True
```

如要微調模型並獲得良好結果，請注意學習率，並使用含有適應期的學習率排程。如下所示：



如果從標準學習率開始，會中斷模型的預先訓練權重。逐步啟動可保留這些資料，直到模型鎖定資料並以合理方式修改為止。斜坡期過後，您可以繼續使用常數或指數衰減學習率。

在 Keras 中，學習率是透過回呼指定，您可以在其中計算每個訓練週期的適當學習率。Keras 會在每個訓練週期將正確的學習率傳遞至最佳化工具。

```
def lr_fn(epoch):
    lr = ...
    return lr

lr_callback = tf.keras.callbacks.LearningRateScheduler(lr_fn, verbose=True)

model.fit(..., callbacks=[lr_callback])
```

解決方案

解決方案筆記本在此。如果遇到困難，可以參考這項工具。



涵蓋內容

- 🧠 深度可分離卷積
- 📊 學習率時間表
- 🐱 微調預先訓練模型。

請花點時間在腦中瀏覽這份檢查清單。

步驟 13 · 約 0 分鐘

13. 恭喜！

您已建構第一個現代卷積類神經網路，並透過 TPU 在短短幾分鐘內完成連續訓練，將準確率提升至 90% 以上。

TPU 實務

TPU 和 GPU 可在 Google Cloud 的 [Vertex AI](#) 上使用：

- 在 [Deep Learning VM](#) 上
- 在 [Vertex AI Notebooks](#) 中
- 在 [Vertex AI Training Jobs](#) 工作中

最後，我們非常重視意見回饋。如果您在本實驗室中發現任何錯誤，或認為有需要改進之處，請告訴我們。你可以透過 GitHub 問題 [[意見回饋連結](#)] 提供意見。



作者：Martin
Görner
Twitter：
[@martin_gorner](#)



TensorFlow